

REGIONAL LANGUAGE QUESTION ANSWERING SYSTEM USING RETRIEVAL-AUGMENTED GENERATION AND LARGE LANGUAGE MODELS

PHASE I REPORT

Submitted by

KABILESH P	230701133
MADHUMITHA P	230701169
MANIKANDAN M	230701174
MANISHAA G	230701176

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this report "**REGIONAL LANGUAGE QUESTION ANSWERING SYSTEM USING RETRIEVAL-AUGMENTED GENERATION AND LARGE LANGUAGE MODELS**" is the bonafide work of **KABILESH P (230701133)**, **MADHUMITHA P (230701169)**, **MANIKANDAN M (230701174)** and **MANISHAA G (230701176)** who carried out the project work under my supervision. Certified further that, to the best of my knowledge, the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates.

This project addresses the following Sustainable Development Goal(s):

Goal No: 4 – Quality Education Goal No: 9 – Industry, Innovation and Infrastructure

Goal No: 10 – Reduced Inequalities Goal No: 16 – Peace, Justice and Strong Institutions

SIGNATURE

Dr. J. Manoranjini , M.E., Ph.D.,

Professor and Head,

Department of Computer Science and
Engineering,

Rajalakshmi Engineering College,
Chennai – 602 105.

SIGNATURE

Dr. Chettiyar Vani. V, M.E., Ph.D.,

Professor,

Department of Computer Science and
Engineering,

Rajalakshmi Engineering College,
Chennai – 602 105.

Submitted to Project Viva-Voce examination held on _____

Internal Examiner

External Examiner

ABSTRACT

Access to reliable information in Indian regional languages remains challenging because of linguistic diversity, limited language-specific resources, and the difficulty of obtaining document-grounded answers. This project presents DesiQuery, a multilingual Retrieval-Augmented Generation (RAG) system for document-based question answering across eight Indian languages. The system accepts PDF, TXT, and DOCX documents, extracts and processes their textual content, divides the content into retrievable chunks, and generates multilingual embeddings using BAAI/bge-m3, which are stored in ChromaDB for semantic retrieval. When a user submits a question, the system retrieves relevant content and applies context selection before passing the evidence to Llama 3 through Ollama for local response generation. The generated response is constrained to the retrieved document evidence and the selected output language, while also retaining source-document, page, chunk, and similarity information for transparency. The system is implemented using React and FastAPI and operates without dependence on cloud-based LLM APIs. An evaluation of 24 test cases across eight languages resulted in 19 PASS, 5 WARNING, and 0 FAIL cases, validating language accuracy, grounding, and source traceability. The results demonstrate the feasibility of a local, document-grounded multilingual QA system while highlighting response completeness and local inference latency as areas for further improvement.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S. MEGANATHAN, B.E, F.I.E.,** our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.,** and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.,** for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.,** our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. J. MANORANJINI, M.E., Ph.D.,** Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **Dr. CHETTIYAR VANI VIVEKANAND, M.E., Ph.D.,** Professor, Department of Computer Science and Engineering for her valuable guidance throughout the course of the project. We thank our Project Coordinator, **Dr. MADHUSUDHANAN , Ph.D.,** Associate Professor, Department of Computer Science and Engineering for his valuable guidance during our review to build our project.

KABILESH P : 230701133
MADHUMITHA P : 230701169
MANIKANDAN M : 230701174
MANISHAA G : 230701176

TABLE OF CONTENTS

Chapter No.	Title	Page No.
	ABSTRACT	iii
	LIST OF TABLES	vii
	LIST OF FIGURES	viii
	LIST OF ABBREVIATIONS	ix
1	INTRODUCTION	1
	1.1 GENERAL	1
	1.2 OBJECTIVES	2
	1.3 EXISTING SYSTEM	2
	1.4 PROPOSED SYSTEM	3
2	LITERATURE SURVEY	5
	2.1 Literature Review	5
3	SYSTEM DESIGN	14
	3.1 SYSTEM ARCHITECTURE	14
	3.2 SYSTEM REQUIREMENTS	17
	3.2.1 Software Requirements	17
	3.2.2 Hardware Requirements	18
	3.3 DOCUMENT PREPROCESSING AND CHUNKING	18
	3.4 QUERY PROCESSING AND EMBEDDING GENERATION	19
	3.5 VECTOR INDEXING AND SEMANTIC RETRIEVAL	20
	3.6 CONTEXT AUGMENTATION AND RESPONSE GENERATION	21
	3.7 SYSTEM EXECUTION FLOW	21

4	PROJECT DESCRIPTION	22
	4.1 INTRODUCTION	22
	4.2 MODULES	
	4.2.1 Document Processing Module	22
	4.2.2 Language Processing and Normalization Module	23
	4.2.3 Embedding and Vector Storage Module	24
	4.2.4 Semantic Retrieval Module	25
	4.2.5 Response Generation Module	25
	4.2.6 Source Document & Page No. Module	26
5	IMPLEMENTATION AND DISCUSSION	27
	5.1 SYSTEM OVERVIEW	27
	5.2 SYSTEM OUTPUT	27
	5.3 RESULT ANALYSIS	30
	5.4 DISCUSSION	33
6	CONCLUSION AND FUTURE WORK	35
	6.1 CONCLUSION	35
	6.2 FUTURE WORK	36
	REFERENCES	37

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
3.1	Software Requirements	17
3.2	Hardware Requirements	18
5.1	Overall Evaluation Results	31
5.2	Language-wise Evaluation Results	32

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
1.1	Overview of the Proposed system	4
3.1	Architecture of the Proposed system	16
3.2	System Execution Flow	21
4.1	Document Processing Module	23
4.2	Language Processing and Normalization Module	24
4.3	Embedding and Vector Storage Module	24
4.4	Semantic Retrieval Module	25
4.5	Response Generation Module	25
5.1	Dashboard Interface	27
5.2	Document Upload and Management Interface	28
5.3	Multilingual Question Answering Interface	28
5.4	Generated Answer with Source Attribution	29
5.5	Retrieved Evidence and AI Pipeline	29
5.6	Document Analyzer Interface	30

LIST OF ABBREVIATIONS

RAG	Retrieval-Augmented Generation
LLM	Large Language Model
QA	Question Answering
NLP	Natural Language Processing
IR	Information Retrieval
IndicNLP	Indic Natural Language Processing (library for Indian languages)
ChromaDB	Open-source vector database used for semantic retrieval
API	Application Programming Interface
FastAPI	Python web framework used for backend services
BGE-M3	Multilingual dense embedding model used for vectorisation
Top-K	The K highest-ranked retrieved results for a query
d	Dimensionality of an embedding vector
$S(q, v)$	Cosine similarity between query vector q and chunk vector v
Wsize	Chunking window size
Woverlap	Chunking window overlap

CHAPTER 1

INTRODUCTION

1.1 GENERAL

The rapid growth of digital information has made question answering (QA) systems an essential tool for accessing knowledge efficiently. However, most existing QA systems are designed primarily for the English language, which creates a significant barrier for users who communicate in Indian regional languages. India is linguistically diverse, with multiple languages differing in script, vocabulary, and grammatical structure, making it difficult to directly apply standard QA systems.

Recent advancements in Large Language Models (LLMs) have improved the ability of machines to understand and generate human language. Despite their success, these models often produce responses that are not factually accurate or are not supported by reliable sources, especially when dealing with domain-specific queries. This issue, commonly referred to as hallucination, reduces the reliability of such systems.

To address these limitations, Retrieval-Augmented Generation (RAG) has emerged as an effective approach. It combines document retrieval with language generation, ensuring that responses are based on actual information rather than purely on learned patterns. This project leverages RAG along with regional language processing techniques to build a system capable of providing accurate, document-based answers.

The proposed system integrates multilingual processing, semantic search, and local large language models to support regional language queries. By grounding responses in a controlled knowledge base, the system aims to improve

accessibility, reliability, and usability for users who prefer Indian regional languages.

1.2 OBJECTIVES OF THE PROJECT

The primary objective of this project is to develop a multilingual, document-grounded question answering system capable of supporting multiple Indian languages. The system is designed to process documents in PDF, TXT, and DOCX formats by extracting and preprocessing textual content, followed by IndicNLP-based normalization to effectively handle regional language variations. It generates multilingual embeddings using the BAAI/bge-m3 model and stores them in ChromaDB to enable efficient semantic retrieval of relevant information. Based on user queries, the system retrieves contextually appropriate document segments and applies context selection before generating accurate, evidence-based responses using Llama 3 through Ollama. Additionally, the system ensures transparency by providing source traceability, including document references and similarity details, and is implemented using a React frontend and FastAPI backend. The entire pipeline is designed to operate locally without reliance on cloud-based LLM APIs, ensuring privacy, control, and offline usability.

1.3 EXISTING SYSTEM

Existing question answering systems are predominantly designed for English and rely heavily on pre-trained language models. These systems often fail to perform effectively when handling regional languages due to limited training data and lack of language-specific processing techniques. As a result, users who communicate in Indian languages face difficulties in accessing accurate information.

Many traditional QA systems depend entirely on large language models without incorporating external knowledge sources. This leads to the problem of hallucination, where the system generates answers that may sound correct but are

not supported by any real data. Such responses can be misleading, especially in critical domains like government schemes and official documents.

Another limitation of existing systems is their inability to process document-based queries efficiently. Most systems do not provide mechanisms for uploading and searching within custom documents, which restricts their usefulness in real-world applications where information is often stored in document form.

Furthermore, existing solutions generally lack support for offline or local deployment. They depend on cloud-based services, raising concerns about data privacy, internet dependency, and system latency. These limitations highlight the need for a more reliable and regionally adaptable solution.

1.4 PROPOSED SYSTEM

The proposed DesiQuery system is a multilingual document-grounded question answering application designed to provide information from selected document sources. The system supports PDF, TXT, and DOCX documents and allows users to select existing documents or upload new documents before submitting a question.

During document processing, the system extracts the available textual content from the uploaded documents. For Indian regional-language content, IndicNLP-based normalization is applied to prepare the extracted text for subsequent processing. The normalized content is then divided into smaller retrievable chunks.

Each document chunk is converted into a multilingual vector representation using BAAI/bge-m3. The generated embeddings, together with the associated document metadata, are stored in ChromaDB.

When a user submits a question, the query is processed and normalized using the language-processing stage and converted into a multilingual embedding using

BAAI/bge-m3. ChromaDB performs semantic retrieval using cosine-based similarity and identifies the most relevant document chunks from the selected knowledge sources.

The retrieved chunks are passed through a context-selection stage. Relevant evidence is retained while redundant or weakly related content may be reduced before response generation.

The selected evidence is supplied to Llama 3 through Ollama for local response generation. The generation process is instructed to use the retrieved document evidence and produce the response in the selected output language. If the required information cannot be identified in the retrieved evidence, the system indicates that the information is unavailable.

The system also retains source-document, page number when available, chunk identifier, and similarity information associated with the retrieved evidence. This information is returned to the frontend so that users can inspect the document evidence associated with an answer.

The system is implemented using React and FastAPI and supports local execution of the core question answering pipeline without dependence on cloud-based LLM APIs.

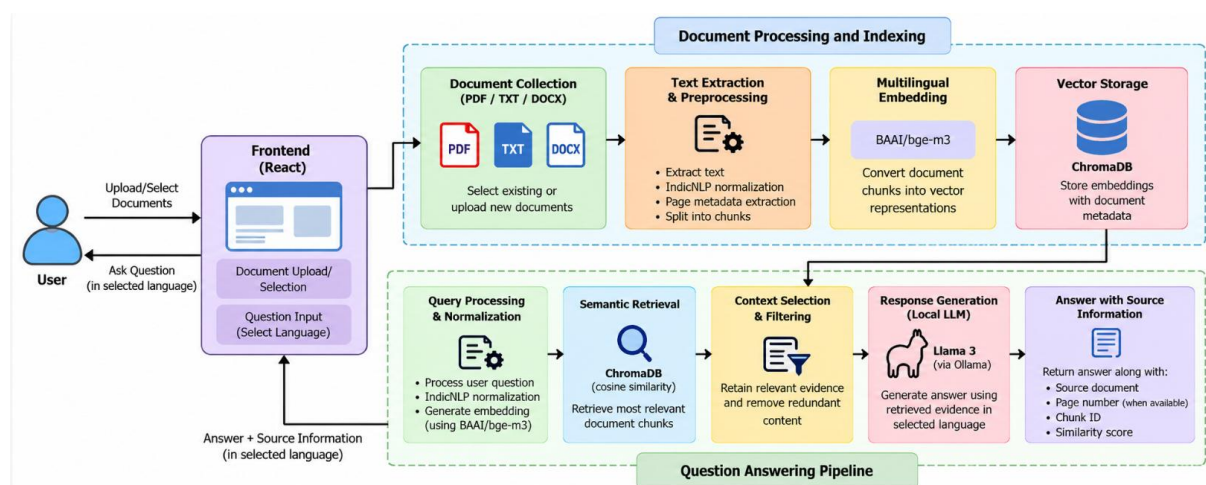


Figure 1.1: Overview of the Proposed System

CHAPTER 2

LITERATURE SURVEY

This section of the paper, provides an all-round review about multiple existing systems that holds similar objective and problem statements as that of the proposed project. The study of these existing papers will produce valuable insights for the construction of the proposed system.

2.1 LITERATURE REVIEW

The paper (Khan et al., 2026), titled “*UQA-RAG: Enhancing Urdu Question-Answer Retrieval with Semantic Similarity and LLMs*,” discusses the application of Retrieval-Augmented Generation for question answering in the Urdu language. The proposed UQA-RAG approach combines dense semantic retrieval using embeddings and ChromaDB with lexical retrieval using BM25. The combination of semantic and lexical retrieval helps the system identify relevant information from the available knowledge sources before generating an answer. The work demonstrates the potential of RAG-based approaches for low-resource languages such as Urdu. However, the effectiveness of the approach is dependent on the availability of suitable language-specific resources and the quality of the retrieved information.

The paper (Nadăs, Diosan, and Tomescu, 2025), titled “*Synthetic Data Generation Using Large Language Models: Advances in Text and Code*,” presents a survey of synthetic data generation using Large Language Models through prompting, self-instruction, and Retrieval-Augmented Generation techniques. The paper discusses the usefulness of synthetic data for applications such as question answering, especially in situations where sufficient real-world training data is unavailable. The study also highlights important challenges associated with synthetic data generation, including factuality, bias, and reliability. Although synthetic data can help increase the amount of available

training information, the quality and correctness of the generated data must be carefully evaluated before using it for QA applications.

The paper (de Oliveira and Cerqueira, 2026), titled “*Resource Constrained Evaluation of Quantized Local LLMs for Retrieval-Augmented Generation on a Reduced-Corpus Setting*,” investigates the performance of locally deployed quantized Large Language Models under resource-constrained RAG environments. The study evaluates different aspects such as retrieval quality, answer quality, latency, and runtime failures. The results indicate that larger language models do not necessarily provide better performance for every RAG application. The study highlights the importance of selecting models according to computational resources, retrieval requirements, and application constraints. This work demonstrates that efficient and smaller models can also be considered for practical RAG-based systems under limited-resource environments.

The paper (Onan et al., 2026), titled “*Comparing ChatGPT, Gemini, and Emerging LLMs in Low-Resource Educational Settings: Reasoning Quality, Consistency, and Explainability*,” evaluates the performance of Large Language Models for educational question answering in low-resource language settings. The study considers different factors such as accuracy, reasoning quality, consistency, and explainability. The research highlights that the performance of LLMs can vary across different languages and that models developed primarily using high-resource languages may not provide the same level of performance for low-resource languages. The study emphasizes the importance of evaluating reasoning and consistency along with simple answer accuracy for multilingual educational QA systems.

The paper (Perevalov et al., 2025), titled “*SPARQL Query Candidate Filtering for Improving the Quality of Multilingual Question Answering over Knowledge Graphs Using Language Models*,” investigates multilingual question answering

over knowledge graphs using language-model-based SPARQL query filtering. The proposed approach focuses on filtering candidate SPARQL queries so that the most relevant query can be selected for answering a user's question. The study explores the use of language models to improve the interaction between natural language questions and structured knowledge graphs across multiple languages. The work demonstrates the potential of combining knowledge graphs and language models for multilingual QA, while also highlighting the challenges associated with language-specific query processing and knowledge representation.

The paper (Sinthussha, Charles, and Weerasinghe, 2025), titled “*Machine Reading Comprehension for the Tamil Language with Translated SQuAD*,” focuses on developing a machine reading comprehension dataset for the Tamil language by translating the SQuAD dataset. The study evaluates multilingual transformer models on the translated Tamil dataset and introduces an answer start correction method to address word-order issues caused during translation. The authors also highlight that conventional answer evaluation metrics may not be completely suitable for Tamil because of its agglutinative linguistic structure. The study therefore emphasizes the importance of language-specific preprocessing and evaluation methods when developing question answering systems for Tamil and other Indian regional languages.

The paper (Amal, Niranjana, and Thushara, 2026), titled “*EduRAG: A Multimodal Explainable RAG System with Deterministic Token Grounding*,” proposes a Retrieval-Augmented Generation approach for educational question answering. The system combines lexical and dense retrieval methods along with query expansion and re-ranking to improve the retrieval of relevant information. It also incorporates a vision-language model and evidence attribution mechanisms to improve explainability and provide supporting evidence for generated answers. The approach demonstrates how multiple retrieval and reasoning components can

be combined to improve the reliability of educational QA systems. However, the integration of multiple modules can increase the complexity and computational requirements of the overall system.

The paper (Behmanesh et al., 2024), titled “*A Novel Open-Domain Question Answering System on Curated and Extracted Knowledge Bases with Consideration of Confidence Scores in Existing Triples,*” presents an open-domain question answering system based on curated and extracted knowledge bases. The approach incorporates confidence scores associated with knowledge-base triples to improve answer selection and ranking. The use of confidence information helps the system determine the reliability of available knowledge before generating an answer. The study demonstrates the importance of knowledge curation and confidence-aware answer ranking in open-domain QA. However, the retrieval performance decreases when queries become complex and contain multiple entities, showing that handling complex questions remains a challenge.

The paper (Liao, Lee, and Wang, 2026), titled “*Cross-Attention Reprogramming for ASR: Bridging Discrete Speech Units and Pretrained Language Models,*” investigates cross-attention reprogramming techniques for improving the interaction between speech representations and pretrained language models. The approach enables pretrained language models to process representations obtained from another modality and supports low-resource and cross-lingual scenarios. The work demonstrates the potential of adapting existing pretrained language models rather than developing completely new models for every application. The study is particularly relevant to multilingual and low-resource environments, although the effectiveness of the approach depends on the quality of the available representations and the target language resources.

The paper (Choi et al., 2025), titled “*Toward Generating Quality Test Questions and Answers Using Quantized Low-Rank Adapters in LLMs*,” investigates the use of Quantized Low-Rank Adaptation for generating questions and answers using Large Language Models. The study demonstrates how QLoRA can be used to adapt models for task-specific question and answer generation while reducing the computational requirements associated with traditional fine-tuning. The approach provides a resource-efficient method for adapting large language models to specific QA tasks. However, the performance of the system still depends on the availability of suitable training data from the target domain.

The paper (Huang and Su, 2025), titled “*TransAlignNet: Semantic Consistency Learning with Multi-Modal Attention for English Translation Inference*,” focuses on improving semantic consistency in multilingual translation-related tasks. The study investigates the use of multi-modal attention mechanisms to preserve the semantic relationship between source and translated information. Maintaining semantic consistency is important in multilingual question answering because translation can introduce changes in word order, meaning, or contextual relationships. The work therefore highlights the importance of preserving semantic information when language models process multilingual content and demonstrates the need for language-aware approaches in cross-lingual NLP applications.

The paper (Ijebu et al., 2025), titled “*Ensemble Answer Selection Leveraging Cross-Lingual Dealignment for Improved QA with Mixture-of-Experts Setup*,” proposes an ensemble-based approach for answer selection in multilingual question answering. The system utilizes cross-lingual dealignment and a mixture-of-experts setup to improve the selection of appropriate answers across different languages. The ensemble architecture allows multiple models or experts to contribute to the final answer selection process. The study demonstrates the potential of combining different model capabilities for multilingual extractive

QA. However, multilingual systems still face challenges due to differences in linguistic structures, available datasets, and resource availability between languages.

The paper (Hambarde and Proença, 2023), titled “*Information Retrieval: Recent Advances and Beyond*,” presents an analysis of recent developments in information retrieval techniques, including sparse retrieval, dense retrieval, hybrid retrieval, and second-stage ranking methodologies. The study emphasizes that information retrieval is a critical component of modern question answering systems because the quality of the retrieved information directly affects the quality of the generated response. The authors highlight that retrieval should be optimized before the generation stage in RAG systems. The study therefore establishes retrieval quality as an important factor in developing reliable question answering systems and indicates that combining different retrieval strategies can improve the overall retrieval process.

The paper (Fields, Chovanec, and Madiraju, 2024), titled “*A Survey of Text Classification with Transformers: How Wide? How Large? How Long? How Accurate? How Expensive? How Safe?*” examines the performance of transformer-based models for text classification and analyzes the relationship between model size, accuracy, computational cost, and safety. The study demonstrates that increasing the size of a language model does not always result in better performance for every natural language processing task. The work highlights the importance of selecting models according to the specific task requirements rather than relying only on model size. These findings are relevant to multilingual QA systems where computational resources, language characteristics, and task-specific performance need to be considered together.

The paper (Hu et al., 2024), titled “*Prompting Large Language Models with Knowledge-Injection for Knowledge-Based Visual Question Answering*,”

investigates the use of knowledge injection to improve knowledge-based visual question answering. The approach provides additional background knowledge to the language model so that it can use external information while answering questions. The study demonstrates that knowledge injection can improve the ability of a model to reason over information that may not be directly available in the input. However, the injected knowledge itself may not always be grounded or reliable. Therefore, the study highlights the importance of verifying external knowledge before incorporating it into the answer generation process.

The paper (Xu et al., 2024), titled “*ChattF: A Knowledge Graph-Enhanced Intelligent Q&A System for Mitigating Factuality Hallucinations in Traditional Folklore*,” proposes a knowledge graph-enhanced question answering system for traditional folklore. The system integrates knowledge graph-based retrieval into a RAG framework to provide supporting information for answer generation. The use of a structured knowledge graph helps the system retrieve factual information before generating an answer and aims to reduce hallucinations. However, the performance of the system depends strongly on the completeness and quality of the knowledge graph. Missing or incomplete information in the knowledge graph can therefore affect the quality of the final answer.

The paper (Saha, Saha, and Malik, 2024), titled “*QuIM-RAG: Advancing Retrieval Augmented Generation with Inverted Question Matching for Enhanced QA Performance*,” proposes a Retrieval-Augmented Generation framework based on inverted question matching. The approach generates hypothetical questions from document fragments to improve the relationship between user queries and the information stored in documents. By associating document fragments with potential questions, the system attempts to improve the retrieval of relevant information during the question answering process. The work demonstrates the potential of question-oriented document processing for

improving retrieval performance and provides an approach for strengthening the connection between user queries and retrieved document content.

The paper (Kong et al., 2025), titled “*Multi-Perspective Consistency Checking for Large Language Model Hallucination Detection: A Black-Box Zero-Resource Approach*,” proposes a multi-perspective consistency checking method for detecting hallucinations in Large Language Models. The approach examines generated responses from different perspectives to identify inconsistencies that may indicate unreliable or hallucinated information. The study highlights the importance of consistency checking because fluent language model responses may still contain unsupported information. The proposed approach demonstrates the need for additional reliability mechanisms in LLM-based question answering systems, particularly when factual correctness is important.

The paper (Ali et al., 2025), titled “*Multilingual Relation Extraction: A Survey*,” provides a survey of multilingual relation extraction and discusses challenges associated with multilingual natural language processing. The study identifies resource imbalance and reproducibility as important challenges in multilingual NLP. Different languages have different amounts of annotated datasets, pretrained resources, and available computational support, which can lead to differences in model performance. The findings are relevant to multilingual question answering because the availability and quality of language-specific resources directly affect information retrieval, language understanding, and answer generation.

The paper (Hindi et al., 2025), titled “*Enhancing the Precision and Interpretability of Retrieval-Augmented Generation (RAG) in Legal Technology: A Survey*,” investigates the application of Retrieval-Augmented Generation in the legal technology domain. The study discusses important challenges associated with retrieval, hallucination, faithfulness, precision, and evaluation in RAG-

based systems. The paper emphasizes that retrieving appropriate and reliable information is essential for generating trustworthy responses, particularly in specialized domains such as law. The study also highlights the importance of evaluating whether generated answers remain faithful to the retrieved evidence. These observations are applicable to multilingual QA systems where reliable retrieval and grounded generation are required.

CHAPTER 3

SYSTEM DESIGN

This chapter of the paper, briefly discusses about the design of the proposed system with the help of Model diagrams. The requirements of the system and deployment are also extensively studied. This chapter also covers the overall architecture of the proposed system along with the overview of modules involved.

3.1 SYSTEM ARCHITECTURE

The proposed system is a web-based Regional Language Question Answering System designed to provide relevant and context-based answers to users who communicate in Indian regional languages. The system integrates regional language processing, document-based knowledge retrieval, Retrieval-Augmented Generation (RAG), and a Large Language Model (LLM) within a single framework. The architecture is designed to ensure that the answers generated by the system are based on the information available in the provided documents rather than relying entirely on the internal knowledge of the language model.

The overall architecture is divided into two major processes: knowledge base construction and question answering. Knowledge base construction is an offline process in which the source documents are collected, processed, divided into meaningful chunks, converted into vector embeddings, and stored in the vector database. Question answering is an online process that begins when a user submits a query through the web interface and continues through language processing, semantic retrieval, context augmentation, and answer generation.

The knowledge base consists of pre-uploaded documents containing the information required by the system to answer user queries. These documents are first subjected to text extraction and preprocessing to remove unnecessary content

and prepare the text for further processing. The processed documents are then divided into smaller, overlapping chunks so that relevant portions of information can be retrieved efficiently. Each chunk is converted into a dense vector representation using the multilingual BGE-M3 embedding model. The generated embeddings, along with the corresponding text and metadata, are stored and indexed in ChromaDB for efficient semantic search.

The user interacts with the system through a React-based frontend, where the user can enter a question in a supported Indian regional language. The query is transferred to the FastAPI backend, which coordinates the different processing stages of the system. The query is first processed using IndicNLP-based language processing, including language detection and text normalization, so that the linguistic characteristics of the regional language are appropriately handled.

After preprocessing, the query is converted into a dense embedding using the same embedding model used for the document chunks. The query embedding is then compared with the stored document embeddings in ChromaDB using cosine similarity. Based on the similarity scores, the most relevant document chunks are retrieved as the top-K results. This retrieval process ensures that the system selects information that is semantically related to the user's question before generating the final response.

The retrieved document chunks are then combined with the user's query to form an enhanced prompt for the Retrieval-Augmented Generation process. This contextual prompt is passed to a locally hosted Large Language Model through Ollama. Instead of generating an answer solely from its pretrained knowledge, the LLM uses the retrieved document content as additional context to generate an answer that is grounded in the system's knowledge base. This approach helps reduce unsupported responses and improves the relevance and factual grounding of the generated answer.

Finally, the generated response is returned from the FastAPI backend to the React frontend and displayed to the user. The separation of the frontend, language processing, document retrieval, vector database, RAG pipeline, and LLM generation components makes the architecture modular, maintainable, and extensible. The knowledge base can also be updated by adding or replacing source documents without requiring changes to the underlying language model, making the system suitable for document-based applications such as government schemes, guidelines, forms, and other official resources.

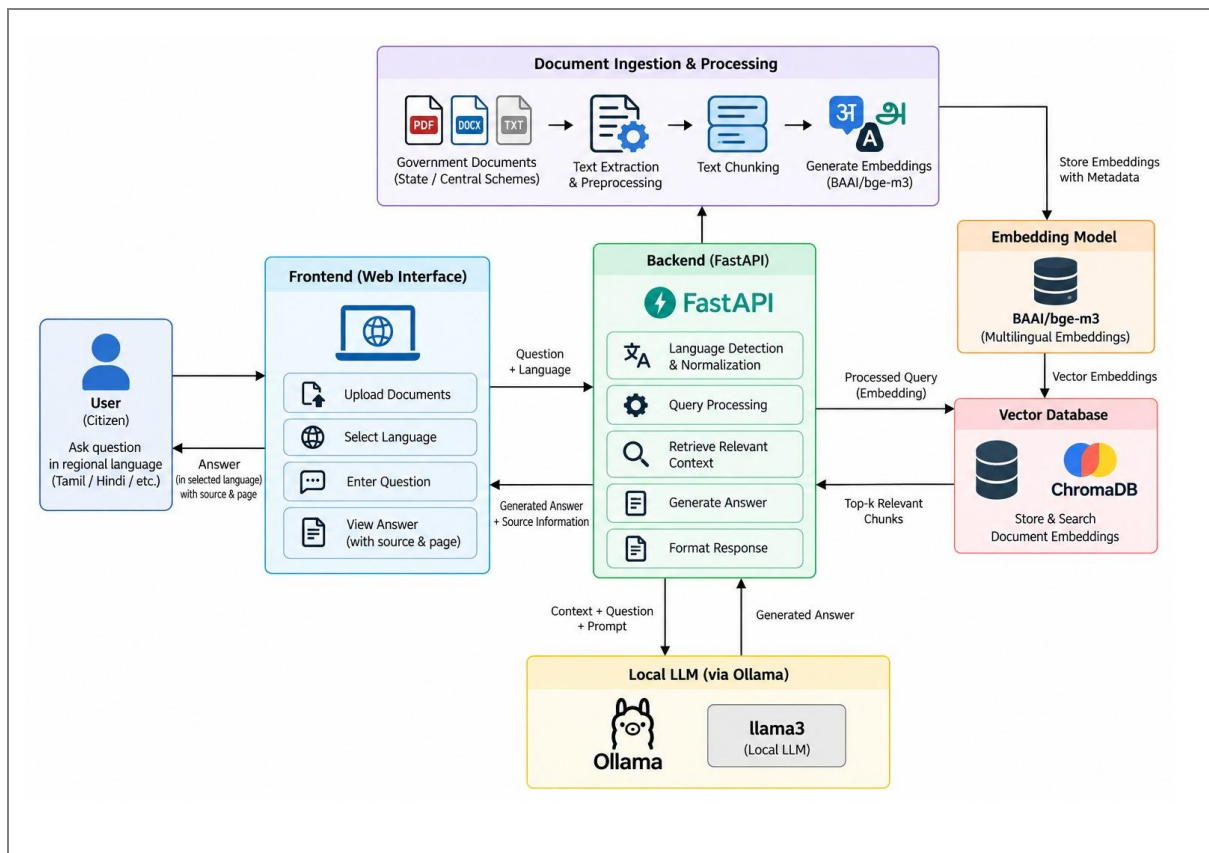


Figure 3.1 Architecture of the Proposed System

3.2 SYSTEM REQUIREMENTS

The system requirements describe the hardware and software specifications necessary for developing and executing the proposed DesiQuery application. The system uses a React-based frontend and a FastAPI backend, integrated with multilingual embedding models, vector storage, and a locally hosted language model. The following requirements support document processing, semantic retrieval, and multilingual question answering.

3.2.1 SOFTWARE REQUIREMENTS

The software requirements specify the tools, frameworks, libraries, and technologies used to develop the proposed system.

Table 3.1 Software Requirements

S. No.	Software / Technology	Purpose
1	Python	Backend development and document processing
2	React.js	Frontend development and user interface
3	FastAPI	Backend API development and communication
4	BAAI/bge-m3	Multilingual embedding generation
5	ChromaDB	Vector storage and semantic retrieval
6	Ollama	Local language model execution

7	Llama 3	Evidence-grounded response generation
8	IndicNLP	Indic-language text normalization
9	HTML, CSS and JavaScript	Frontend structure, styling and functionality
10	Visual Studio Code	Development environment

3.2.2 HARDWARE REQUIREMENTS

Table 3.2 Hardware Requirements

S. No.	Hardware Component	Requirement
1	Processor	Multi-core Intel or AMD processor
2	RAM	Minimum 8 GB; 16 GB or above recommended
3	Storage	Sufficient free disk space for models, dependencies and documents
4	GPU	Optional; compatible GPU can improve inference performance
5	Internet Connection	Required for initial model and dependency downloads; not required for core QA operation after setup

3.3 DOCUMENT PREPROCESSING AND CHUNKING

The system accepts PDF, TXT, and DOCX documents as knowledge sources.

The uploaded document is first processed using the corresponding text

extraction mechanism. For PDF documents, page information is retained whenever it can be reliably obtained.

After text extraction, IndicNLP-based normalization is applied to Indian regional-language content. This normalization stage standardizes Unicode-based Indic text and prepares it for subsequent processing. The normalized content is then divided into smaller retrievable chunks.

Let the document collection be represented as:

$$\mathbf{D} = \{\mathbf{d}_1, \mathbf{d}_2, \dots, \mathbf{d}_N\}$$

The processed documents are divided into chunks:

$$\mathbf{C} = \{\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_m\}$$

where $\mathbf{c}_j$ represents a retrievable portion of a source document.

Each chunk is associated with available metadata such as:

- Source document name
- Page number (when available)
- Chunk identifier

The resulting chunks are then passed to the BAAI/bge-m3 embedding stage.

The preprocessing pipeline operates on the characters obtained from the document extraction stage. Therefore, the quality of the extracted text directly affects all subsequent stages, including normalization, embedding, retrieval, and response generation.

3.4 QUERY PROCESSING AND EMBEDDING GENERATION

When the user submits a question Q , the query is processed according to the selected language. IndicNLP-based normalization is applied to prepare Indian regional-language input for semantic representation.

The normalized query is represented as:

$$Q' = \text{IndicNLP_Normalize}(Q)$$

The normalized query is then converted into a dense multilingual embedding using BAAI/bge-m3:

$$q = E(Q')$$

Similarly, each document chunk c_j is represented as:

$$v_j = E(c_j)$$

where $E(\cdot)$ represents the BAAI/bge-m3 embedding function, q is the query vector, and v_j is the vector representation of the j -th document chunk.

Both the query and document chunks are represented in a multilingual semantic space, enabling retrieval of semantically related content even when the query wording does not exactly match the document text.

The generated query embedding is then supplied to the ChromaDB semantic retrieval stage.

3.5 VECTOR INDEXING AND SEMANTIC RETRIEVAL

The chunk vectors v_j are stored and indexed in ChromaDB. When a query embedding q is received, the similarity between q and each stored chunk vector v_j is computed using cosine similarity:

$$S(q, v_j) = (q \cdot v_j) / (\|q\| \|v_j\|)$$

The top- k relevant chunks R_k are then retrieved by selecting the chunks with the highest similarity scores:

$$R_k = \arg \max_{j \in \{1, \dots, m\}^{\wedge k}} S(q, v_j)$$

3.6 CONTEXT AUGMENTATION AND RESPONSE GENERATION

The selected chunks R_k are concatenated with the user's normalised query Q' to form an enhanced prompt P :

$$P = \text{PromptFormat}(Q', R_k)$$

This prompt P is supplied to a local LLM available through Ollama, which produces the final response A : $A = \text{LLM_Ollama}(P)$

The FastAPI backend then returns the response A to the React frontend for display to the user.

3.7 SYSTEM EXECUTION WORKFLOW

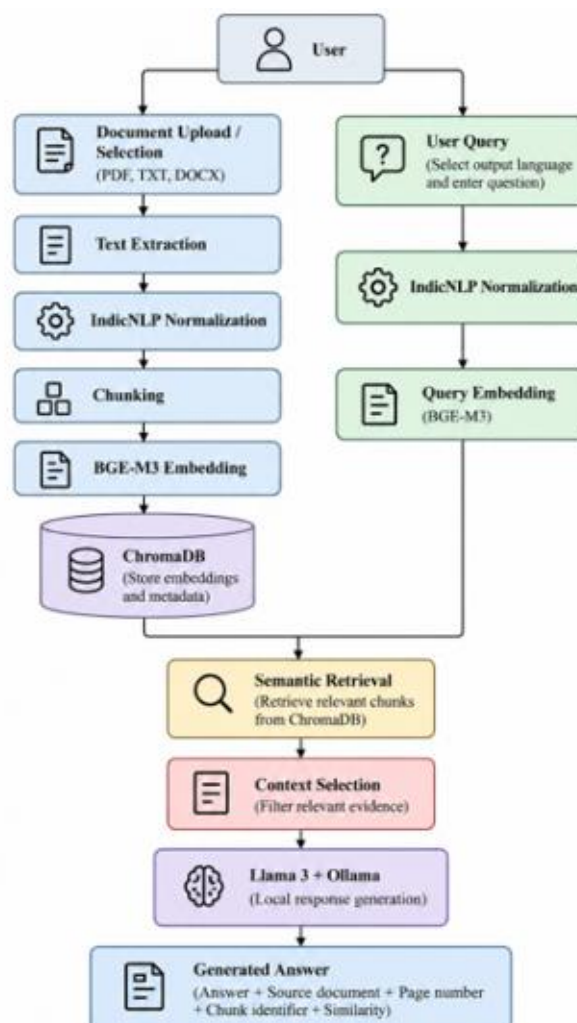


Figure 3.2: System Execution Flow

CHAPTER 4

PROJECT DESCRIPTION

4.1 METHODOLOGY

The methodology of DesiQuery follows a Retrieval-Augmented Generation (RAG) approach to provide evidence-based multilingual question answering. The system extracts and preprocesses text from selected documents, applies language normalization, divides the content into chunks, and generates multilingual embeddings using BAAI/bge-m3. These embeddings are stored in ChromaDB for semantic retrieval. When a user submits a question, the system retrieves relevant chunks and provides them as context to a locally hosted Llama 3 model through Ollama to generate an answer in the selected output language, along with available source document and page details.

4.2 MODULES

This describes the various modules of the proposed Regional Language Question Answering System. The system is divided into multiple functional modules to ensure efficient handling of document processing, regional language understanding, semantic retrieval, and response generation. Each module performs a specific task, and together they contribute to the overall functionality and performance of the system.

4.2.1 Document Collection and Preprocessing Module

The Document Collection and Preprocessing Module manages PDF, TXT, and DOCX documents used as knowledge sources. Users can either select available documents or upload their own documents through the frontend interface.

The uploaded document is processed to extract its available textual content. For PDF documents, page information is retained whenever it can be reliably

obtained. Following extraction, IndicNLP-based normalization is applied to supported Indian-language Unicode text.

The normalized content is then prepared for subsequent chunking and embedding, while relevant document metadata is preserved for later source attribution.

The quality of the extracted text plays a critical role in the effectiveness of downstream stages in the pipeline. Documents containing scanned images or incorrectly encoded legacy-font text may not yield usable Unicode text through standard extraction methods. Such cases require Optical Character Recognition (OCR) or dedicated legacy-font conversion techniques, which are beyond the scope of the current implementation.

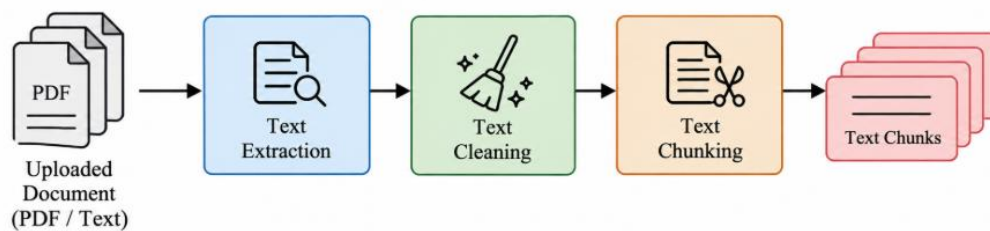


Figure 4.1 Document Processing Module

4.2.2 Language Processing and Normalization Module

The Language Processing and Normalization Module prepares Indian-language document text and user queries for subsequent semantic processing. IndicNLP is used to normalize supported Indian-language Unicode text after document extraction and before query embedding.

The normalization process helps standardize textual representations and prepares the content for multilingual embedding generation. The module operates on the text obtained from the document extraction stage and does not perform optical character recognition (OCR) or legacy font conversion.

For user queries, the selected language is retained, and the query is normalized prior to conversion into a multilingual embedding using BAAI/bge-m3.

Thus, this module functions as the language normalization layer between text extraction or query input and the semantic embedding process.

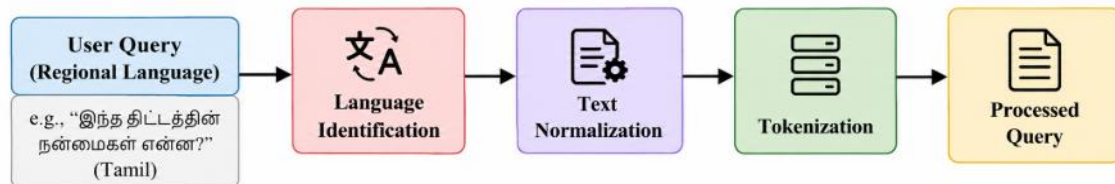


Figure 4.2 Language Processing and Normalization Module

4. 2.3 Embedding and Vector Storage Module

The embedding and vector storage module converts the processed document chunks and user queries into dense vector representations. These embeddings capture the semantic meaning of the text, enabling context-based matching instead of simple keyword search.

The generated embeddings are stored in a vector database, which acts as the system's knowledge base. Efficient indexing and storage of embeddings allow fast and accurate retrieval of relevant information during query processing.

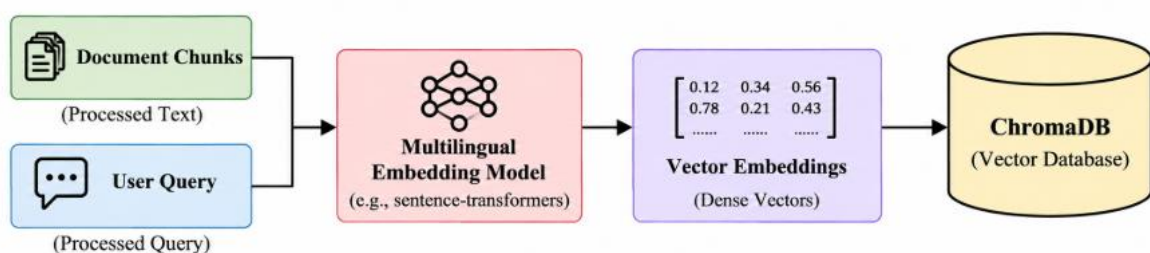


Figure 4.3 Embedding and Vector Storage Module

4.2.4 ChromaDB And Semantic Retrieval Module

The semantic retrieval module is responsible for identifying relevant document content based on the user query. The query is converted into an embedding and compared with stored document embeddings using similarity measures.

The system retrieves the most relevant document chunks based on similarity scores. This ensures that the information used for generating responses is contextually appropriate and aligned with the user's query.

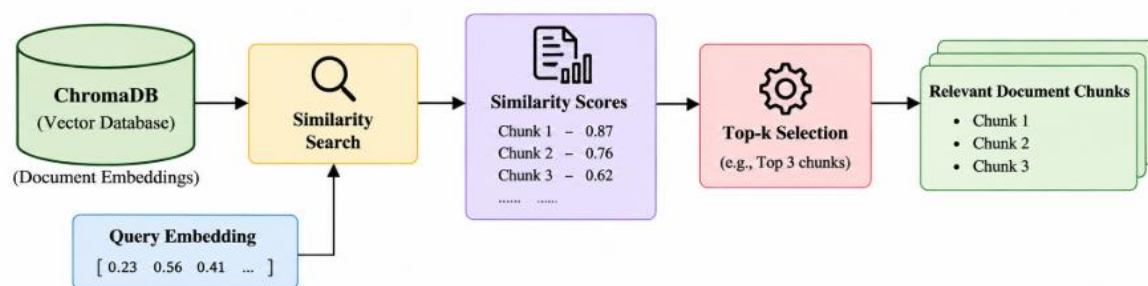


Figure 4.4 Semantic retrieval Module

4.2.5 RAG, Ollama And Local LLM Module

This module combines the user query with the selected retrieved evidence to construct the context for response generation. Llama 3 through Ollama generates the answer locally, with instructions to use the retrieved document information and follow the selected output language.

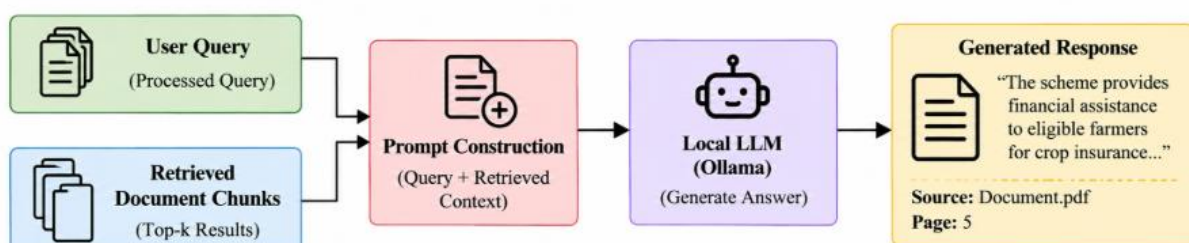


Figure 4.5 Response Generation Module

4.2.6 Source Document & Page No. Module

This module maintains and returns the source information associated with retrieved chunks. It provides available document names, page numbers, chunk identifiers, similarity scores, and retrieved evidence so that users can inspect the information supporting an answer.

CHAPTER 5

IMPLEMENTATION AND DISCUSSION

5.1 SYSTEM OVERVIEW

This chapter presents the developed DesiQuery application and its outputs. It illustrates the major user interfaces, document management process, multilingual question-answering functionality, retrieved evidence, source attribution, and document analysis features. The chapter also presents the system evaluation results and discusses the observations and limitations identified during testing.

5.2 SYSTEM OUTPUT

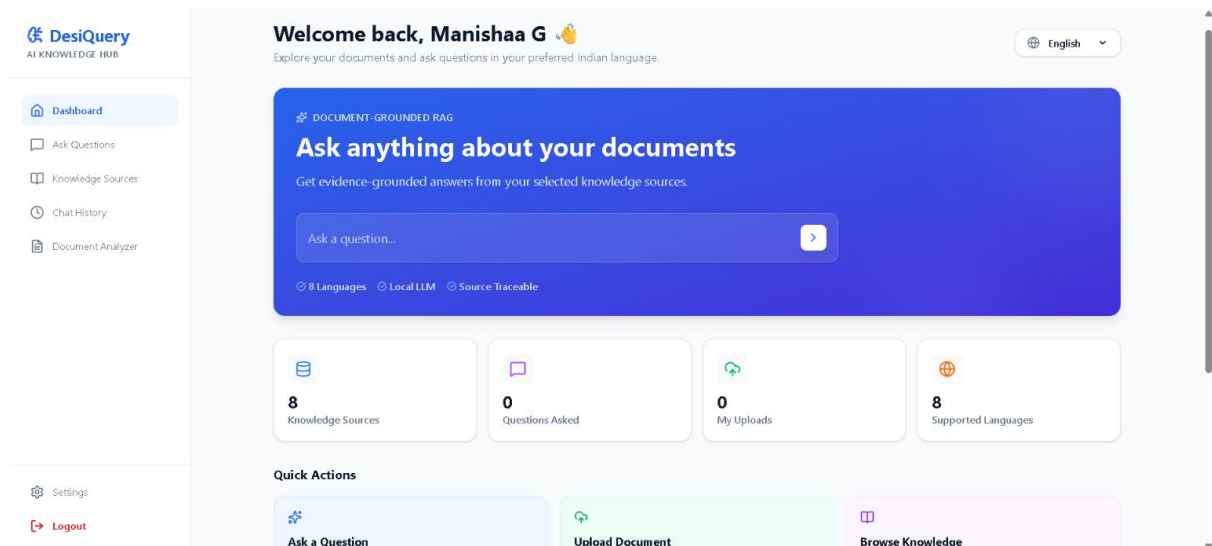


Figure 5.1: Dashboard Interface

The dashboard provides the main interface for accessing the DesiQuery application. It allows users to navigate between document management, multilingual question answering, and document analysis functionalities.

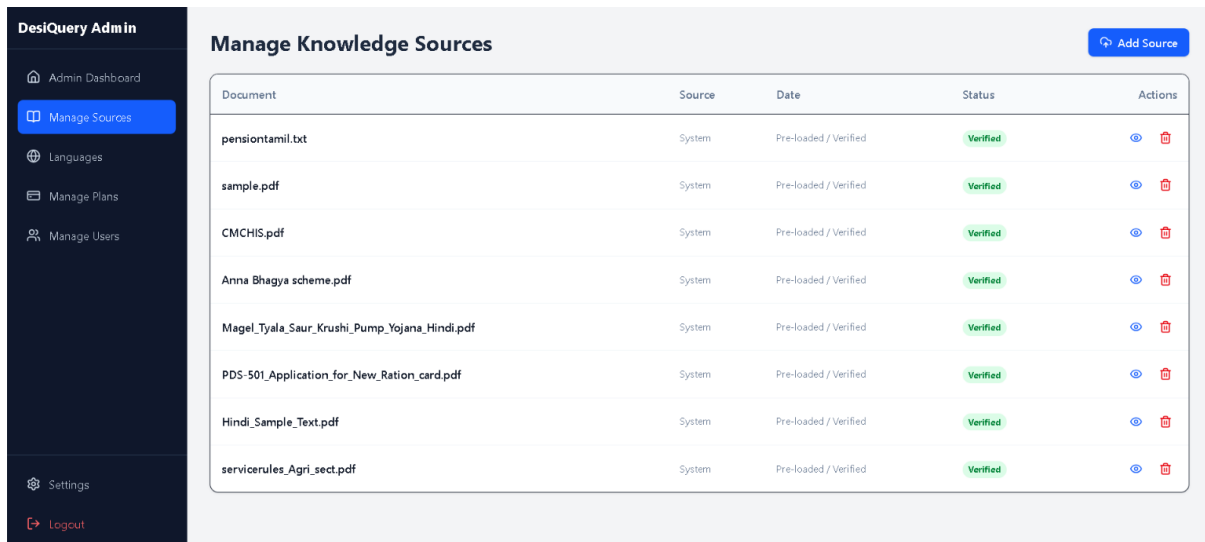


Figure 5.2: Document Upload and Management Interface

This interface allows users to upload PDF, TXT, and DOCX documents and manage the available knowledge sources. Users can select the documents they want the system to use when answering questions.

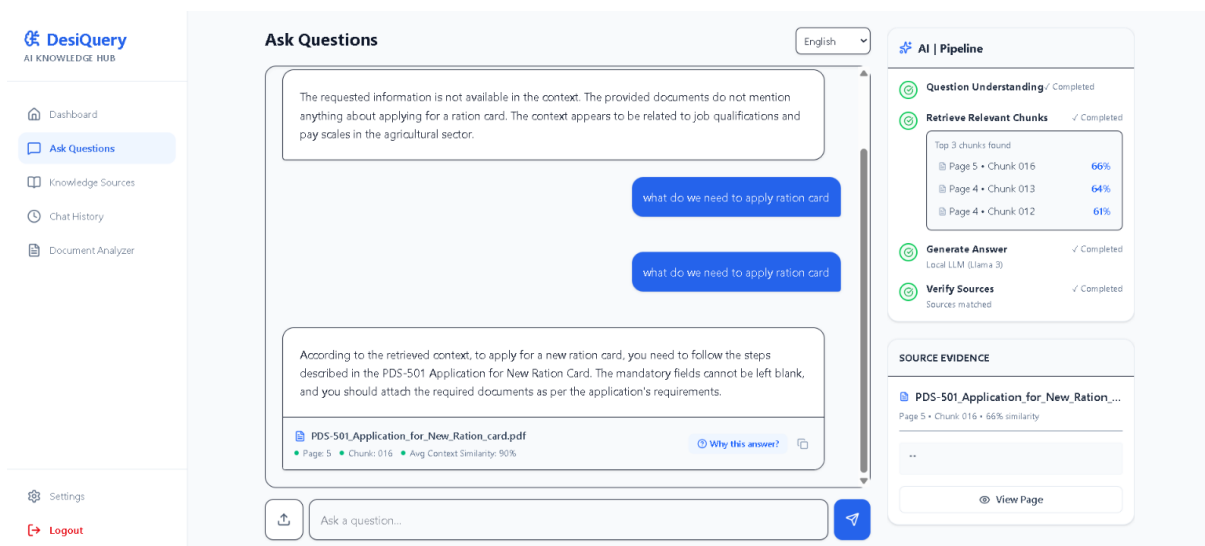


Figure 5.3: Multilingual Question Answering Interface

The question-answering interface allows users to enter questions in the supported languages and select their preferred output language. The submitted query is processed using the selected documents to retrieve relevant information.

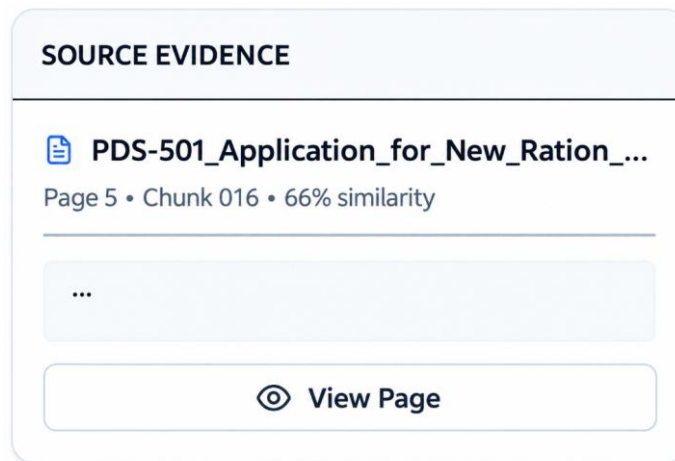


Figure 5.4: Generated Answer with Source Attribution

This output displays the generated response along with the available source document, page number, and retrieval information. It helps users identify the document evidence associated with the generated answer.

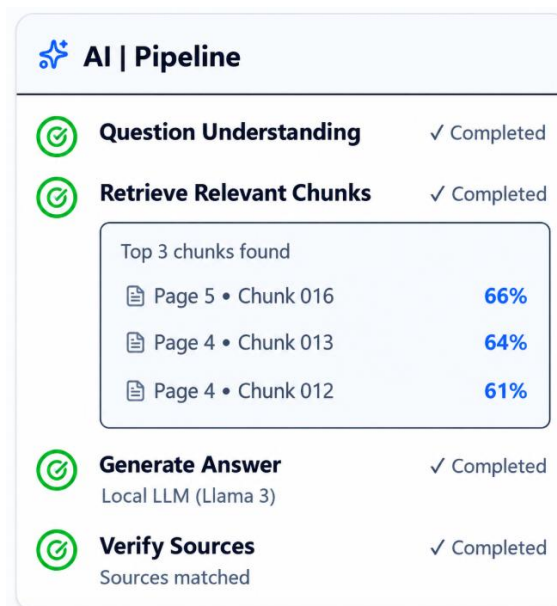


Figure 5.5: Retrieved Evidence and AI Pipeline

This interface displays the retrieved document chunks and their similarity scores, along with information about the processing pipeline. It helps users inspect the evidence selected by the retrieval stage before response generation.

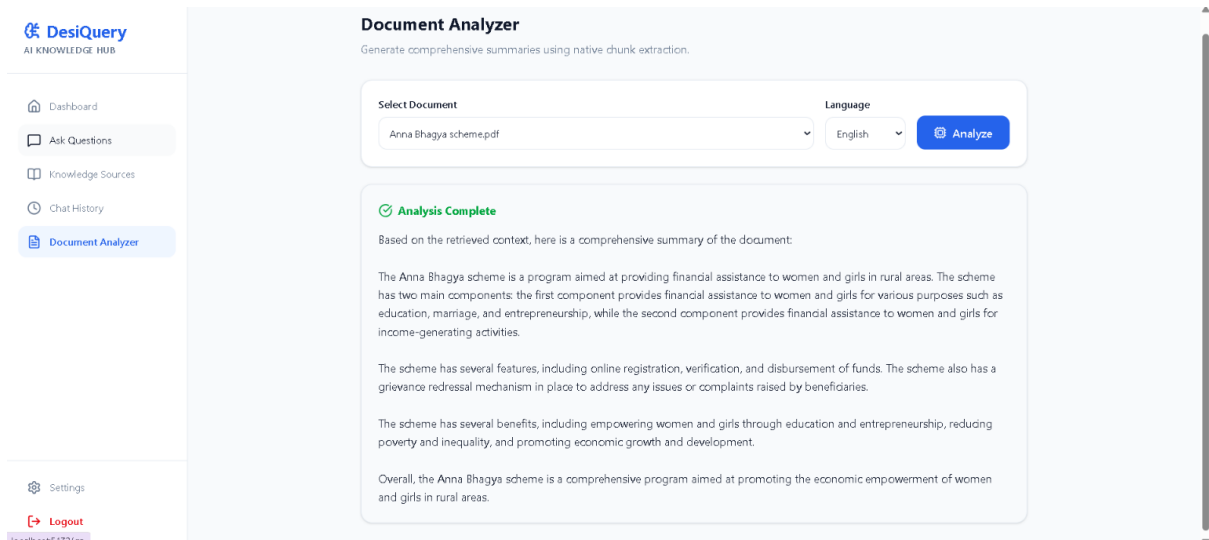


Figure 5.6: Document Analyzer Interface

The document analyzer provides an interface for examining uploaded documents and viewing document-related information. It supports users in understanding the contents of their selected knowledge sources.

5.3 RESULT ANALYSIS

The implemented system was evaluated using 24 test cases across the eight supported languages: English, Tamil, Hindi, Telugu, Kannada, Malayalam, Bengali, and Marathi.

Three types of questions were included:

1. Procedure-based questions
2. Application/document-related questions
3. Unsupported questions

Each category contained one test case for each of the eight languages.

The evaluation produced 19 PASS, 5 WARNING, and 0 FAIL cases.

TABLE 5.1 OVERALL EVALUATION RESULTS

Evaluation Measure	Result
Total test cases	24
PASS	19 (79.2%)
WARNING	5 (20.8%)
FAIL	0 (0%)
Language validation	100%
Grounding validation	100%
Source traceability	100%
Average response time	≈102 seconds

Language validation achieved 100% across the evaluated test cases, indicating that the generated responses followed the selected output language according to the implemented validation procedure.

Grounding validation also achieved 100% for the evaluated test set. The evaluation verified whether relevant claims, numerical information, and procedural content could be supported by the retrieved document evidence. This result represents grounding validation for the evaluated cases and should not be interpreted as general factual accuracy.

Source traceability achieved 100% across the evaluated cases. The system retained the source document and, when available, page number, chunk identifier, retrieval similarity, and retrieved content.

The five WARNING cases were associated with response-completeness limitations in some multilingual outputs. No evaluated case was classified as a FAIL.

The average end-to-end response time was approximately 102 seconds under the tested local hardware and configuration. Local LLM generation contributed most of the observed latency, whereas semantic retrieval generally required substantially less time.

TABLE 5.2 LANGUAGE-WISE EVALUATION RESULTS

Language	PASS	WARNING	FAIL
English	3	0	0
Tamil	3	0	0
Hindi	3	0	0
Telugu	1	2	0
Kannada	2	1	0
Malayalam	1	2	0
Bengali	3	0	0
Marathi	3	0	0
Total	19	5	0

The results show that all evaluated languages satisfied the language-validation requirement. The warning cases occurred in Telugu, Kannada, and Malayalam and were associated with response completeness rather than grounding or source-traceability failure.

5.4 DISCUSSION

The implemented DesiQuery system demonstrates the feasibility of document-grounded question answering across the eight supported Indian languages under the evaluated configuration. The use of BAAI/bge-m3 enables multilingual semantic representation, while ChromaDB provides local similarity-based retrieval of relevant document content.

The context-selection stage further limits the information supplied to the language model by retaining relevant retrieved content and reducing unnecessary context. This is useful for local inference because it controls the amount of information supplied to Llama 3 during generation.

The source-traceability mechanism provides additional transparency by associating generated responses with retrieved document information. Users can inspect the source document, page number, chunk identifier, similarity value, and retrieved content when these values are available.

The evaluation achieved complete validation for language consistency, grounding, and source traceability across the tested cases. However, five warning cases indicate that response completeness remains an area for improvement for some multilingual outputs.

Response latency is another limitation of the current implementation. Local Llama 3 inference contributes substantially to the total processing time. The measured latency is dependent on the available hardware, model configuration, context size, and generated output length.

The evaluation is based on a relatively small set of 24 test cases. Therefore, the results characterize the implemented system under the tested conditions and should not be interpreted as generalized performance across all regional-language questions and document collections.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

DesiQuery was developed as a document-grounded multilingual Question Answering system for accessing information from documents in Indian regional languages. The system integrates document processing, multilingual semantic representation, vector-based retrieval, context selection, and local Large Language Model generation into a single application.

The implemented system uses BAAI/bge-m3 to generate multilingual embeddings, ChromaDB for semantic retrieval, and Llama 3 through Ollama for local response generation. The application supports PDF, TXT, and DOCX documents and enables users to select the documents from which information should be retrieved. This document-selection mechanism helps restrict the question-answering process to the intended knowledge sources.

The system supports eight languages: English, Tamil, Hindi, Telugu, Kannada, Malayalam, Bengali, and Marathi. Questions are processed using multilingual semantic retrieval, and the generated response is produced according to the selected output language. The response-generation process is constrained using the retrieved document evidence so that information unavailable in the selected documents is not intentionally generated from external model knowledge.

A major feature of the system is source traceability. Along with the generated answer, the application provides available source-document information, page number, chunk identifier, retrieval similarity, and retrieved evidence. This allows users to inspect the document content associated with an answer and provides greater transparency than presenting a generated response alone.

The system was evaluated using 24 test cases covering three categories of questions across the eight supported languages. The evaluation produced 19 PASS cases, 5 WARNING cases, and 0 FAIL cases. Language validation, grounding validation, and source traceability achieved 100% validation across the evaluated test set. The evaluation also showed that local LLM generation is the primary contributor to the observed response latency.

The warning cases indicate that response completeness can still be improved for certain multilingual responses. In addition, the relatively small evaluation dataset and dependence on local hardware limit the extent to which the reported results can be generalized to larger document collections and broader language use cases.

Overall, the developed system demonstrates the feasibility of combining multilingual semantic retrieval with locally deployed LLM-based generation for document-grounded question answering. The integration of retrieval evidence and source metadata provides a practical approach for improving transparency while maintaining a fully local QA pipeline.

6.2 FUTURE WORK

Future development of DesiQuery can focus on expanding the evaluation dataset and supporting additional Indian regional languages. Retrieval performance can be improved through hybrid search and advanced ranking techniques, while local LLM inference can be optimized using quantized models and efficient configurations. Document processing can be extended to support scanned PDFs, tables, images, and legacy-font conversion. Further improvements can include adaptive context selection, enhanced source highlighting, and more comprehensive performance evaluation to improve the system's usability and reliability.

REFERENCES

1. S. Amal, A. R. Niranjana and M. G. Thushara (2026) 'EduRAG: A Multimodal Explainable RAG System with Deterministic Token Grounding'.
2. M. Ali, R. Speck, H. M. Zahera, M. Saleem, D. Moussallem and A.-C. N. Ngomo (2025) 'Multilingual Relation Extraction: A Survey', IEEE Access.
3. S. Behmanesh, A. Talebpour, M. Shamsfard and M. M. Jafari (2024) 'A Novel Open-Domain Question Answering System on Curated and Extracted Knowledge Bases with Consideration of Confidence Scores in Existing Triples'.
4. J. Choi, S. Hong, S. Hong, J. Park and E.-S. Jung (2025) 'Toward Generating Quality Test Questions and Answers Using Quantized Low-Rank Adapters in LLMs'.
5. M. L. L. de Oliveira and J. de Jesus Fiais Cerqueira (2026) 'Resource-Constrained Evaluation of Quantized Local LLMs for Retrieval-Augmented Generation on a Reduced-Corpus Setting', IEEE Access.
6. M. Nadas, L. Diosan and A. Tomescu (2025) 'Synthetic Data Generation Using Large Language Models: Advances in Text and Code', IEEE Access.
7. J. Fields, K. Chovanec and P. Madiraju (2024) 'A Survey of Text Classification with Transformers: How Wide? How Large? How Long? How Accurate? How Expensive? How Safe?'
8. K. A. Hambarde and H. Proença (2023) 'Information Retrieval: Recent Advances and Beyond'.
9. M. Hindi, L. Mohammed, O. Maaz and A. Alwarafy (2025) 'Enhancing the Precision and Interpretability of Retrieval-Augmented Generation (RAG) in Legal Technology: A Survey', IEEE Access.
10. S. Huang and T. Su (2025) 'TransAlignNet: Semantic Consistency Learning with Multi-Modal Attention for English Translation Inference'.
11. F. F. Ijebu, Y. Liu, C. Sun, N. Jere, I. D. Mienye and P. U. Usip (2025) 'Ensemble Answer Selection Leveraging Cross-Lingual Dealignment for Improved QA with Mixture-of-Experts Setup'.
12. Z. Hu, P. Yang, F. Liu, Y. Meng and X. Liu (2024) 'Prompting Large Language Models with Knowledge-Injection for Knowledge-Based Visual Question Answering', Big Data Mining and Analytics, Vol.7, No.3.
13. T. F. Khan, M. Arslan, F. Alturise, A. H. Almutairi and M. T. Raza (2026) 'UQA-RAG: Enhancing Urdu Question-Answer Retrieval with Semantic Similarity and LLMs'.

14. L. Kong, X. Zhong, J. Chen, H. Fu and Y. Wang (2025) 'Multi-Perspective Consistency Checking for Large Language Model Hallucination Detection: A Black-Box Zero-Resource Approach', *Frontiers of Information Technology & Electronic Engineering*.
15. P.-J. Liao, H.-Y. Lee and H.-M. Wang (2026) 'Cross-Attention Reprogramming for ASR: Bridging Discrete Speech Units and Pretrained Language Models'.
16. A. Onan, A. H. Nasution, T. Celikten and P. Cetin (2026) 'Comparing ChatGPT, Gemini, and Emerging LLMs in Low-Resource Educational Settings: Reasoning Quality, Consistency, and Explainability', *IEEE Access*.
17. A. Perevalov, A. Gashkov, M. Eltsova and A. Both (2025) 'SPARQL Query Candidate Filtering for Improving the Quality of Multilingual Question Answering over Knowledge Graphs Using Language Models', *Journal of Web Engineering*, Vol.24.
18. B. Saha, U. Saha and M. Z. Malik (2024) 'QuIM-RAG: Advancing Retrieval-Augmented Generation with Inverted Question Matching for Enhanced QA Performance', *IEEE Access*, Vol.12.
19. A. V. A. Sinthussha, E. Y. A. Charles and R. Weerasinghe (2025) 'Machine Reading Comprehension for the Tamil Language with Translated SQuAD'.
20. J. Xu, H. Zhang, H. Zhang, J. Lu and G. Xiao (2024) 'ChatTF: A Knowledge Graph-Enhanced Intelligent Q&A System for Mitigating Factuality Hallucinations in Traditional Folklore', *IEEE Access*, Vol.12.

DesiQuery: An Evidence-Grounded Multilingual Framework for Regional Language Question Answering

Chettiyar Vani Vivekanand

*Department of Computer
Science and Engineering
Rajalakshmi Engineering College
Chennai, India*

chettiyarvanivivekanand@rajalakshmi.edu.in

Kabilesh P

*Department of Computer
Science and Engineering
Rajalakshmi Engineering College
Chennai, India*

230701133@rajalakshmi.edu.in

Manishaa G

*Department of Computer
Science and Engineering
Rajalakshmi Engineering College
Chennai, India*

230701176@rajalakshmi.edu.in

Manikandan M

*Department of Computer
Science and Engineering
Rajalakshmi Engineering College
Chennai, India*

230701174@rajalakshmi.edu.in

Madhumitha P

*Department of Computer
Science and Engineering
Rajalakshmi Engineering College
Chennai, India*

230701169@rajalakshmi.edu.in

Abstract—Access to reliable digital information in Indian regional languages remains challenging due to linguistic diversity, limited language-specific resources, and the tendency of large language models (LLMs) to generate unsupported information when domain-specific evidence is unavailable. The developed DesiQuery framework allows users to select or upload documents and ask questions in eight Indian languages, where BAAI/bge-m3 is used to identify semantically relevant document chunks in ChromaDB and Llama 3 generates responses through Ollama using the retrieved evidence. To improve reliability and transparency, the system restricts answer generation to retrieved evidence and provides source-document, page-level, chunk-level, and similarity information when available. Queries for which relevant information cannot be retrieved are explicitly identified rather than being answered using unsupported model knowledge. The system was evaluated using a multilingual test suite covering supported and unsupported information-seeking queries across the eight target languages, with evaluation focused on language consistency, grounding, source traceability, response completeness, and latency. The results demonstrate the feasibility of locally deployed, source-traceable RAG for regional-language document question answering while also identifying response-generation limitations for some longer multilingual outputs. The proposed framework provides a privacy-oriented and extensible approach for accessing document-based information in Indian regional languages without dependence on cloud-based LLM APIs.

Index Terms—Regional Language Processing, Question Answering, Retrieval-Augmented Generation, Large Language Models, IndicNLP, ChromaDB, Semantic Retrieval, Ollama, Natural Language Processing.

I. INTRODUCTION

An era of rapidly expanding digital information has increased the use of Question Answering (QA) systems for retrieving information from large document collections. However, reliable document-based QA remains challenging for Indian regional languages due to differences in vocabulary,

scripts, morphology, and syntax. The challenge becomes more significant when answers must be derived from specific documents such as government, administrative, or domain-specific resources.

Large Language Models (LLMs) exhibit great capabilities for language understanding and generation; however, these models may produce information which is not part of the documents provided. The challenge of extracting the relevant external sources and then feeding the LLM with these sources can be addressed by employing Retrieval-Augmented Generation (RAG). Multilingual QA requires an appropriate semantic representation across the languages for efficient retrieval and generation of grounded response. In some instances, local deployment may be necessary because of the sensitivity of the information in the documents; local execution of the retrieval and language model pipeline ensures greater control over these issues. To address these challenges, we present DesiQuery, a locally deployed multilingual document-grounded QA system based on Retrieval-Augmented Generation.

This proposed system combines multilingual semantic representations using BAAI/bge-m3, ChromaDB for vector storage and semantic retrieval, and Llama 3 with Ollama for local response generation. The system supports eight Indian languages and constrains generated responses to the retrieved document evidence. It also maintains source-document, page, chunk, and similarity metadata, allowing users to inspect the evidence associated with each generated answer. If the required information is not available in the selected documents, the system indicates that the information is unavailable rather than generating an unsupported response.

The main contributions of this work are:

- 1) **Multilingual document-grounded QA:** A multilingual question-answering system is developed for document-

grounded question answering across eight Indian languages, enabling semantic retrieval and response generation in regional languages.

- 2) **Local QA with source traceability:** A streamlined local QA workflow is developed using BAAI/bge-m3, ChromaDB, and Llama 3 through Ollama. The system incorporates source document, page, chunk, and similarity metadata to provide evidence traceability for generated answers.
- 3) **Evidence-grounded evaluation:** The complete pipeline is evaluated using both supported and unsupported queries across the supported languages, considering language consistency, grounding, source traceability, response completeness, and response latency.

II. RELATED WORKS

Recent development of LLMs has brought enhancement in the capability of QA system for natural language understanding and knowledge-based response generation. But when the information is in a specialized document, the responses created by LLM can be hard to verify. RAG tackles this by extracting relevant data prior to generating a response. Khan et al. [1] suggested UQA-RAG for Urdu QA system that involved lexical and semantic retrieval, and Amal et al. [2] introduced another deterministic token-based grounding-based system named EduRAG for Urdu QA. Hindi et al. [3] discussed the use of RAG in legal domain, Xu et al. [4] introduced ChatTf with knowledge graphs for factuality and Saha et al. [5] presented QuIM-RAG by inverse question matching. Retrieval and grounding play an important role in these studies, but they concentrate primarily on single retrieval methods or retrieval areas, and not on a multi-lingual regional-language QA, local inference, and source-level traceability.

A. Multilingual and Regional-Language Question Answering

Multiple languages in a QA system pose difficulties. This is because of the differences in word usage, grammar and writing patterns. In the context Onan et al. [6] present a collection of low-resource educational applications. Reviewed the performance of cross-modal LLM QA, focusing on quality Logical and consistent answers and explainability. The work of Perevalov et al. [7] addressed the question of QA on multilingual knowledge graphs and explored the use of language models to filter candidate queries. Language-specific regional-language QA has also been investigated. datasets. Sinthusua et al. [8] were concerned with Tamil machine reading. This involves improving comprehension by making a dataset from the SQuAD corpus more accessible by translating it. Their study showed some of the challenges of implementing QA approaches. between different languages that have different linguistic features. Ali et al. [9] reviewed multilingual relation extraction and discussed challenges associated with uneven resource availability and multilingual representation.

These studies motivate the development of language-aware retrieval and evaluation approaches for regional-language QA

systems, particularly for languages with limited resources and distinct linguistic structures.

B. Semantic Retrieval and Multilingual Representation

Dense retrieval uses embeddings to encode queries and documents and matches them by semantic relevance. Hambarde and Proença [10] examined sparse, dense, and hybrid information retrieval approaches and emphasized the importance of retrieval and ranking quality.

Multilingual embedding models enable representations from multiple languages to be mapped into a shared semantic space, supporting cross-lingual retrieval. DesiQuery uses BAAI/bge-m3 embeddings for multilingual semantic matching, which is stored in the local vector repository called ChromaDB.

However, retrieving a relevant document chunk alone does not guarantee that the generated response will remain faithful to the retrieved evidence. Therefore, semantic retrieval must be combined with evidence grounding and response verification to improve the reliability of multilingual QA.

C. LLM Hallucination and Response Reliability

Hallucination and unsupported generation remain significant limitations of LLM-based QA systems. Hu et al. [11] explored how to incorporate outside knowledge into VQA and highlighted the importance of grounded information. A multi-perspective consistency method to identify hallucinations in LLM was introduced by Kong et al. [12]. Behmanesh et al. [13] investigated confidence-based answer selection for knowledge-based question answering.

These studies demonstrate that improving the language model alone is not sufficient to achieve reliable QA. Retrieval quality, evidence verification, and grounding of generated answers are also important. The proposed DesiQuery system addresses these challenges by restricting response generation to retrieved document evidence and retaining source and page metadata for traceability.

D. Local and Resource-Constrained LLM Deployment

Local deployment introduces additional constraints related to model size, inference time, memory requirements, and available computational resources. Cerqueira [14] conducted a quantitative evaluation of quantized local LLMs for RAG under resource-constrained settings, considering answer quality, retrieval performance, and runtime behavior. Choi et al. [15] investigated the generation of quality test questions and answers using quantized low-rank adapters in LLMs. Liao et al. [16] explored cross-attention reprogramming for automatic speech recognition by connecting discrete speech units with pretrained language models.

These studies illustrate the importance of efficient deployment and model adaptation under resource constraints. In contrast, DesiQuery uses BAAI/bge-m3 embeddings, local ChromaDB retrieval, and Llama 3 through Ollama without requiring fine-tuning or cloud-hosted generation. This architecture enables the document knowledge base to be updated without retraining the language model.

TABLE I
COMPARATIVE ANALYSIS OF DESIQUERY AND EXISTING WORKS

Study	Main Focus	Retrieval Knowledge Method	Language Domain	Local Generation	Sourceability	Traceability
Khan al. [1]	et Urdu Question Answering	Lexical and semantic retrieval	Urdu	Not the primary focus	Limited	
Amal al. [2]	et Explainable educational QA	Multimodal RAG and token grounding	Educational	Not the primary focus	Grounding-based	
Hindi al. [3]	et RAG in legal technology	Retrieval-based generation	Legal domain	Not the primary focus	Interpretability-focused	
Xu al. [4]	et Factuality-aware QA	Knowledge graph enhanced retrieval	Traditional folklore	Not the primary focus	Knowledge-graph based	
Saha al. [5]	et Retrieval improvement	Inverted question matching	General QA	Not the primary focus	Not the primary focus	
Cerqueira et al. [14]	et Local RAG deployment	Retrieval with local LLM	Resource-constrained setting	Yes	Not the primary focus	
DesiQuery	Regional-language document QA	BGE-M3 ChromaDB	+ Eight Indian languages	Llama 3 + Ollama	Document, page, chunk and similarity	

The existing research has paved the way for RAG, Multilingual QA, Semantic Retrieval, Hallucination Detection, and Local LLM Deployment, but they are studied in isolation or in limited domains. DesiQuery integrates multilingual BAAI/bge-m3 embeddings, retrieval with ChromaDB, Llama 3 (with Ollama) and source-level information for document QA in regional language. System evaluation is conducted over eight Indian languages for both supported and unsupported queries, based on language consistency, language grounding, source traceability, response completeness and latency.

III. PROPOSED SYSTEM

DesiQuery is a multilingual, document-grounded question-answering framework designed for local deployment. It provides answers in eight Indian languages while ensuring that responses are generated only from the information contained in the selected documents. Users can select pre-loaded documents or upload PDF, TXT, and DOCX files and submit queries in any supported language. The system architecture is divided into modular stages covering document processing, language handling, semantic retrieval, response generation, and source attribution.

The developed system consists of six major modules: (1) Document Collection and Preprocessing, (2) Language Processing and Normalization, (3) Document Chunking and BGE-M3 Embedding, (4) ChromaDB and Semantic Retrieval, (5) RAG with Ollama and a Local LLM, and (6) Source Document and Page Number Tracking. The modular design allows individual components to be replaced, inspected, and diagnosed independently without duplicating their responsibilities.

A. Document Collection and Preprocessing

The system accepts PDF, TXT, and DOCX documents that may either be pre-loaded or uploaded by the user. During ingestion, text is extracted from the selected documents and associated with the corresponding document name and page number whenever available. Documents containing insufficient extractable text, such as image-only PDFs, are rejected during the ingestion stage. The preserved document and page metadata are subsequently used during retrieval and source attribution.

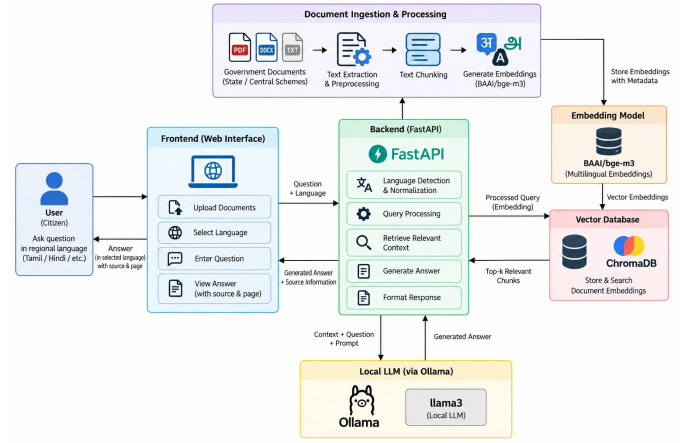


Fig. 1. System Architecture

B. Language Processing and Normalization

DesiQuery supports eight different Indian Regional languages. The user selects the language before submitting a query, and the selected language is used throughout the query processing and answer generation stages. The generated response is also expected to remain consistent with the selected language. Language consistency is subsequently validated during system evaluation.

C. Document Chunking and BGE-M3 Embeddings

The extracted document text is segmented into smaller chunks to enable fine-grained semantic retrieval. Each chunk retains its associated document identifier, chunk identifier, and page information whenever available. The BAAI/bge-m3 embedding model is used to generate multilingual semantic representations of both document chunks and user queries. The resulting embeddings are stored along with their metadata for subsequent retrieval.

D. ChromaDB and Semantic Retrieval

ChromaDB is used as the local vector database for storing document embeddings and associated metadata. When a user submits a query, its semantic representation is generated using the same embedding model and compared with the indexed document representations. The most relevant chunks are retrieved and ranked according to their semantic similarity. The retrieved evidence and associated metadata are retained for context construction and source attribution.

E. RAG and Local LLM Generation

The retrieved evidence is supplied as context to Llama 3 through Ollama for evidence-constrained generation. The generation process is constrained to the retrieved document evidence and the selected output language. If the required information is not present in the retrieved evidence, the system indicates that sufficient information is not available rather than generating unsupported content.

Context selection is performed before generation to retain the most relevant evidence while controlling the amount of information provided to the local language model. This method is used to minimize unsupported generation by selecting the most pertinent retrieved evidence for the generation of a response.

F. Source Document and Page Number Tracking

The system maintains source information associated with the retrieved chunks, including the source document, page number, chunk identifier, and retrieval similarity score when available. The generated answer is returned together with its supporting source information, allowing users to inspect the evidence used for answer generation. The similarity score is presented as a retrieval indicator and is not interpreted as a probability or confidence score.

G. Overall Processing Pipeline

The overall processing pipeline begins when the user selects or uploads documents through the Document Upload interface and initiates text extraction. The extracted content is processed, normalized, divided into chunks, and converted into BGE-M3 embeddings. The embeddings are then indexed in ChromaDB for semantic retrieval. When a query is submitted, its embedding is generated and compared with the indexed document representations. Relevant evidence is selected and supplied to Llama 3 through Ollama for grounded answer generation. Finally, the answer is returned together with the corresponding source attribution.

The complete processing flow is represented as:

Document Upload → Text Extraction → Language Processing
 → Chunking → BGE-M3 Embedding → ChromaDB
 → Semantic Retrieval → Context Selection
 → Llama 3/Ollama → Answer + Source Attribution

This modular architecture separates document preparation, semantic retrieval, response generation, and evidence traceability, enabling pipeline-level monitoring and diagnosis of individual processing stages.

IV. METHODOLOGY

The document grounded multilingual QA system, Desi Query, is a retrieval + generation based approach that employs locally hosted LLM.

Desi Query:

- ingests content in eight languages (English, Tamil, Hindi, Telugu, Malayalam, Marathi, Bengali, Kannada),
- normalizes and chunks it,
- applies multilingual embedding and,
- performs vector storage and semantic retrieval,
- selects context from the retrieved data,
- generates answer using locally hosted LLM, validates answer to language of concern, and,
- attributes source to generated answer.

A. Document processing chunking:

Desi Query accepts PDF, TXT, and DOCX documents. Supported PDF files are retained in page granularity if page number information can be retrieved reliably from the file. Text-only documents with very limited content are ignored. Content collection D is represented as

$$D = \{d_1, d_2, \dots, d_n\}$$

Document chunks are stored for future retrieval,

$$d_i \rightarrow \{ci_1, ci_2, \dots, ci_m\}$$

Chunk ID, Page Number are preserved to support source attribution with output chunks.

B. Embedding semantic retrieval:

Both document chunks and queries are transformed to vectors using BAAI/bge-m3,

$$e_i = f_{\text{BGE-M3}}(c_i), \quad e_q = f_{\text{BGE-M3}}(q)$$

Vectors are stored in ChromaDB, and chunks similar to query are identified by cosine similarity:

$$\text{sim}(q, c_i) = \frac{e_q \cdot e_i}{\|e_q\| \|e_i\|}$$

Ranking based on this measurement yields the evidence set. Sentence based filtering to remove redundant information is performed to improve quality of the content before text generation.

C. Retrieval-augmented generation:

Contextually selected evidence is combined with query and fed to Llama 3 via Ollama:

$$A = G(q, C'_q)$$

where A is answer, q is question, C'_q is filtered context, and G is the locally hosted LLM. Llama is instructed to respond in the language of interest, use only provided evidence, avoid unsupported statements and, communicate when information cannot be found in documents.

D. Validating attributing output:

The output is validated for content and language, and context source information, including document name, page number, chunk ID, and similarity score, is provided with the answer. In particular, similarity scores are indicators of retrieval relevance, rather than probabilities of answer correctness. When evidence for answering a question is not available, the answer is reported as "answer not available". In addition, generation truncation flag is observed and, in case of a truncated answer, a new prompt with a larger maximum output token count is sent to Llama.

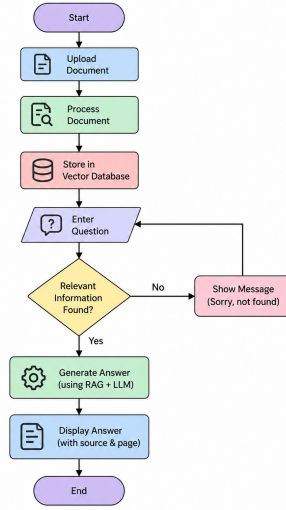


Fig. 2. Algorithm Workflow

V. EXPERIMENTAL SETUP AND RESULTS

A. Experimental Setup

The DesiQuery system was evaluated to assess its performance in document-grounded multilingual question answering. The evaluation focused on language consistency, evidence grounding, source traceability, response completeness, and response latency.

A total of 24 test cases were constructed using three question categories: procedure-based questions, application/document-related questions, and unsupported questions. Each category contained one test case for each of the eight supported languages. Thus, each language was evaluated using three test cases.

The evaluation was performed through the FastAPI query interface using the complete processing pipeline, including query embedding, semantic retrieval with ChromaDB, context selection, and local Llama 3 generation through Ollama. Retrieval metadata was also verified to ensure that generated responses could be traced to the retrieved document evidence.

TABLE II
MULTILINGUAL EVALUATION TEST SET

Category	EN	TA	HI	TE	KN	ML	BN	MR	Total
Procedure	1	1	1	1	1	1	1	1	8
Application/Documentation	1	1	1	1	1	1	1	1	8
Unsupported	1	1	1	1	1	1	1	1	8
Total	3	3	3	3	3	3	3	3	24

B. Evaluation Criteria

Five criteria were considered during evaluation. Language consistency checks whether the response follows the selected output language. Grounding assesses whether statements, numerical details, and procedural instructions in the response can be traced to the selected document content. Source traceability verifies the consistency of document, chunk, and page metadata when available. Response completeness checks

whether the answer terminates naturally within the configured generation limits. Response latency indicates the time taken to process a query and generate the corresponding answer.

The evaluation was implemented using automated validation together with diagnostic information produced by the backend. Grounding validation specifically measures support from the retrieved evidence and is not treated as a general measure of factual accuracy.

C. Results and Analysis

The evaluation produced 19 PASS, 5 WARNING, and 0 FAIL results across the 24 test cases. Language validation, grounding validation, and source traceability achieved successful validation for all evaluated cases. The warning cases were associated primarily with response completion under the current local generation configuration.

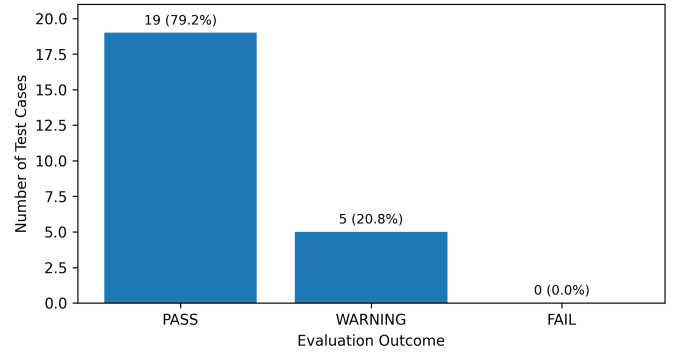


Fig. 3. Overall Evaluation Results

The language-wise results are presented in Table III. English, Tamil, Hindi, Bengali, and Marathi produced PASS results for all three test cases. Kannada produced two PASS and one WARNING, while Telugu and Malayalam produced one PASS and two WARNING results each. These warning cases indicate that response completeness can vary for some regional-language outputs under the current local LLM configuration. They should be interpreted within the scope of the evaluated test set and not as a general ranking of language performance.

TABLE III
LANGUAGE-WISE EVALUATION RESULTS

Language	PASS	WARNING	FAIL	Total
English	3	0	0	3
Tamil	3	0	0	3
Hindi	3	0	0	3
Telugu	1	2	0	3
Kannada	2	1	0	3
Malayalam	1	2	0	3
Bengali	3	0	0	3
Marathi	3	0	0	3
Total	19	5	0	24

The retrieval diagnostics showed that semantic retrieval generally required substantially less time than local LLM

generation. Most observed retrieval operations were completed within approximately 0.24–0.63 s, with one measured retrieval time of 8.10 s. The retrieved chunks were retained for inspection, while context selection reduced unnecessary or weakly relevant content before generation. A formal Recall@K or Mean Reciprocal Rank value is not reported because the current evaluation set does not contain explicit relevance labels required for these metrics.

The latency measurements further showed that local LLM inference was the dominant contributor to end-to-end response time. The observed first-generation inference times in the available diagnostic measurements ranged from approximately 52.63 to 84.58 s, while the average total response time across the current 24-test evaluation was approximately 102 s. These measurements are specific to the local hardware and inference configuration and should not be considered universal performance values. The results suggest that the local LLM inference constitutes the larger portion of the end-to-end latency in the current implementation, and semantic retrieval typically takes much less time.

D. Reliability and Discussion

The system can handle unsupported queries, incomplete/corrupted documents, generation failures, invalid metadata, and incomplete responses. When the information cannot be found in the selected documents, the system does not fall back to unsupported model knowledge, but reports the information is not available. The evaluation results demonstrate language fidelity, grounding, and source traceability across the supported eight languages, although additional work is needed to enhance response completeness for Telugu, Kannada, and Malayalam.

The main bottleneck in performance is LLM inference within the local inference engine. This evaluation used only 24 test cases and performance is hardware-dependent, so these results reflect the system’s performance in the tested environment, but not generally for the full set of regional-language queries and document types.

VI. CONCLUSION AND FUTURE SCOPE

DesiQuery is a locally deployed multilingual RAG framework for Document Grounded Question Answering across 8 Indian languages, built using BAAI/bge-m3, ChromaDB, Ollama model of Llama 3, along with source, page, chunk and similarity traceability. The evaluation of 24 test cases resulted in 19 PASS, 5 WARNING and 0 FAIL cases, with successful language, grounding and source-traceability validation for the test set. The results validate the proposed workflow and identify the local LLM inference latency and limitations in response completion as areas for future enhancements.

Future work will focus on expanding the multilingual evaluation dataset, creating formally annotated retrieval benchmarks, and conducting ablation studies for retrieval depth and context filtering. Further improvements will investigate optimized or quantized local models to reduce inference latency, improved processing of scanned PDFs, tables and complex

document structures, and language-specific evaluation and optimization for regional-language responses. These extensions can improve the retrieval reliability, multilingual robustness, and computational efficiency of DesiQuery.

REFERENCES

- [1] T. F. Khan, M. Arslan, F. Alturise, A. H. Almutairi, and M. T. Raza, “Uqa-rag: Enhancing urdu question-answer retrieval with semantic similarity and llms,” *IEEE Access*, vol. 14, pp. 2868–2882, 2026.
- [2] S. Amal, A. R. Niranjana, and M. G. Thushara, “Edurag: A multimodal explainable rag system with deterministic token grounding,” *IEEE Access*, vol. 14, pp. 90 849–90 859, 2026.
- [3] M. Hindi, L. Mohammed, O. Maaz, and A. Alwarafy, “Enhancing the precision and interpretability of retrieval-augmented generation (rag) in legal technology: A survey,” *IEEE Access*, vol. 13, pp. 46 171–46 189, 2025.
- [4] J. Xu, H. Zhang, H. Zhang, J. Lu, and G. Xiao, “Chattf: A knowledge graph-enhanced intelligent q&a system for mitigating factuality hallucinations in traditional folklore,” *IEEE Access*, vol. 12, pp. 162 638–162 650, 2024.
- [5] B. Saha, U. Saha, and M. Zubair Malik, “Quim-rag: Advancing retrieval-augmented generation with inverted question matching for enhanced qa performance,” *IEEE Access*, vol. 12, pp. 185 401–185 410, 2024.
- [6] A. Onan, A. H. Nasution, T. Celikten, and P. Cetin, “Comparing chatgpt, gemini, and emerging llms in low-resource educational settings: Reasoning quality, consistency, and explainability,” *IEEE Access*, vol. 14, pp. 32 807–32 838, 2026.
- [7] A. Perevalov, A. Gashkov, M. Eltsova, and A. Both, “Sparql query candidate filtering for improving the quality of multilingual question answering over knowledge graphs using language models,” *Journal of Web Engineering*, vol. 24, no. 4, pp. 563–592, 2025.
- [8] A. Vijeevaraj Ann Sinthusha, E. Y. A. Charles, and R. Weerasinghe, “Machine reading comprehension for the tamil language with translated squad,” *IEEE Access*, vol. 13, pp. 13 312–13 328, 2025.
- [9] M. Ali, R. Speck, H. M. Zahera, M. Saleem, D. Moussallem, and A.-C. Ngonga Ngomo, “Multilingual relation extraction: A survey,” *IEEE Access*, vol. 13, pp. 151 907–151 933, 2025.
- [10] K. A. Hambarde and H. Proença, “Information retrieval: Recent advances and beyond,” *IEEE Access*, vol. 11, pp. 76 581–76 604, 2023.
- [11] Z. Hu, P. Yang, F. Liu, Y. Meng, and X. Liu, “Prompting large language models with knowledge-injection for knowledge-based visual question answering,” *Big Data Mining and Analytics*, vol. 7, no. 3, pp. 843–857, 2024.
- [12] L. Kong, X. Zhong, J. Chen, H. Fu, and Y. Wang, “Multi-perspective consistency checking for large language model hallucination detection: A black-box zero-resource approach,” *Frontiers of Information Technology & Electronic Engineering*, vol. 26, no. 11, pp. 2298–2309, 2025.
- [13] S. Behmanesh, A. Talebpour, M. Shamsfard, and M. Mahdi Jafari, “A novel open-domain question answering system on curated and extracted knowledge bases with consideration of confidence scores in existing triples,” *IEEE Access*, vol. 12, pp. 160 741–160 760, 2024.
- [14] M. L. Lima de Oliveira and J. de Jesus Fiais Cerqueira, “Resource-constrained evaluation of quantized local llms for retrieval-augmented generation on a reduced-corpus setting,” *IEEE Access*, vol. 14, pp. 70 402–70 422, 2026.
- [15] J. Choi, S. Hong, S. Hong, J. Park, and E.-S. Jung, “Toward generating quality test questions and answers using quantized low-rank adapters in llms,” *IEEE Access*, vol. 13, pp. 87 793–87 809, 2025.
- [16] P.-J. Liao, H.-Y. Lee, and H.-M. Wang, “Cross-attention reprogramming for asr: Bridging discrete speech units and pretrained language models,” *IEEE Access*, vol. 14, pp. 662–678, 2026.

Dr Vani

T043_PAPER .pdf

 Rajalakshmi Engineering College

Document Details

Submission ID

trn:oid::3618:153500330

Submission Date

Oct 2, 2026, 1:07 PM GMT+5:30

Download Date

Oct 2, 2026, 1:14 PM GMT+5:30

File Name

T043_PAPER .pdf

File Size

617.8 KB

6 Pages

4,268 Words

25,917 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

DesiQuery: An Evidence-Grounded Multilingual Framework for Regional Language Question Answering

Chettiyar Vani Vivekanand

*Department of Computer
Science and Engineering
Rajalakshmi Engineering College
Chennai, India*

chettiyarvanivivekanand@rajalakshmi.edu.in

Kabilesh P

*Department of Computer
Science and Engineering
Rajalakshmi Engineering College
Chennai, India*

2307011333@rajalakshmi.edu.in

Manishaa G

*Department of Computer
Science and Engineering
Rajalakshmi Engineering College
Chennai, India*

230701176@rajalakshmi.edu.in

Manikandan M

*Department of Computer
Science and Engineering
Rajalakshmi Engineering College
Chennai, India*

230701174@rajalakshmi.edu.in

Madhumitha P

*Department of Computer
Science and Engineering
Rajalakshmi Engineering College
Chennai, India*

230701169@rajalakshmi.edu.in

Abstract—Access to reliable digital information in Indian regional languages remains challenging due to linguistic diversity, limited language-specific resources, and the tendency of large language models (LLMs) to generate unsupported information when domain-specific evidence is unavailable. The developed DesiQuery framework allows users to select or upload documents and ask questions in eight Indian languages, where BAAI/bge-m3 is used to identify semantically relevant document chunks in ChromaDB and Llama 3 generates responses through Ollama using the retrieved evidence. To improve reliability and transparency, the system restricts answer generation to retrieved evidence and provides source-document, page-level, chunk-level, and similarity information when available. Queries for which relevant information cannot be retrieved are explicitly identified rather than being answered using unsupported model knowledge. The system was evaluated using a multilingual test suite covering supported and unsupported information-seeking queries across the eight target languages, with evaluation focused on language consistency, grounding, source traceability, response completeness, and latency. The results demonstrate the feasibility of locally deployed, source-traceable RAG for regional-language document question answering while also identifying response-generation limitations for some longer multilingual outputs. The proposed framework provides a privacy-oriented and extensible approach for accessing document-based information in Indian regional languages without dependence on cloud-based LLM APIs.

Index Terms—Regional Language Processing, Question Answering, Retrieval-Augmented Generation, Large Language Models, IndicNLP, ChromaDB, Semantic Retrieval, Ollama, Natural Language Processing.

I. INTRODUCTION

An era of rapidly expanding digital information has increased the use of Question Answering (QA) systems for retrieving information from large document collections. However, reliable document-based QA remains challenging for Indian regional languages due to differences in vocabulary,

scripts, morphology, and syntax. The challenge becomes more significant when answers must be derived from specific documents such as government, administrative, or domain-specific resources.

Large Language Models (LLMs) exhibit great capabilities for language understanding and generation; however, these models may produce information which is not part of the documents provided. The challenge of extracting the relevant external sources and then feeding the LLM with these sources can be addressed by employing Retrieval-Augmented Generation (RAG). Multilingual QA requires an appropriate semantic representation across the languages for efficient retrieval and generation of grounded response. In some instances, local deployment may be necessary because of the sensitivity of the information in the documents; local execution of the retrieval and language model pipeline ensures greater control over these issues. To address these challenges, we present DesiQuery, a locally deployed multilingual document-grounded QA system based on Retrieval-Augmented Generation.

This proposed system combines multilingual semantic representations using BAAI/bge-m3, ChromaDB for vector storage and semantic retrieval, and Llama 3 with Ollama for local response generation. The system supports eight Indian languages and constrains generated responses to the retrieved document evidence. It also maintains source-document, page, chunk, and similarity metadata, allowing users to inspect the evidence associated with each generated answer. If the required information is not available in the selected documents, the system indicates that the information is unavailable rather than generating an unsupported response.

The main contributions of this work are:

- 1) **Multilingual document-grounded QA:** A multilingual question-answering system is developed for document-

grounded question answering across eight Indian languages, enabling semantic retrieval and response generation in regional languages.

- 2) **Local QA with source traceability:** A streamlined local QA workflow is developed using BAAI/bge-m3, ChromaDB, and Llama 3 through Ollama. The system incorporates source document, page, chunk, and similarity metadata to provide evidence traceability for generated answers.
- 3) **Evidence-grounded evaluation:** The complete pipeline is evaluated using both supported and unsupported queries across the supported languages, considering language consistency, grounding, source traceability, response completeness, and response latency.

II. RELATED WORKS

Recent development of LLMs has brought enhancement in the capability of QA system for natural language understanding and knowledge-based response generation. But when the information is in a specialized document, the responses created by LLM can be hard to verify. RAG tackles this by extracting relevant data prior to generating a response. Khan et al. [1] suggested UQA-RAG for Urdu QA system that involved lexical and semantic retrieval, and Amal et al. [2] introduced another deterministic token-based grounding-based system named EduRAG for Urdu QA. Hindi et al. [3] discussed the use of RAG in legal domain, Xu et al. [4] introduced ChatTf with knowledge graphs for factuality and Saha et al. [5] presented QuIM-RAG by inverse question matching. Retrieval and grounding play an important role in these studies, but they concentrate primarily on single retrieval methods or retrieval areas, and not on a multi-lingual regional-language QA, local inference, and source-level traceability.

A. Multilingual and Regional-Language Question Answering

Multiple languages in a QA system pose difficulties. This is because of the differences in word usage, grammar and writing patterns. In the context Onan et al. [6] present a collection of low-resource educational applications. Reviewed the performance of cross-modal LLM QA, focusing on quality Logical and consistent answers and explainability. The work of Perevalov et al. [7] addressed the question of QA on multilingual knowledge graphs and explored the use of language models to filter candidate queries. Language-specific regional-language QA has also been investigated. datasets. Sinthusha et al. [8] were concerned with Tamil machine reading. This involves improving comprehension by making a dataset from the SQuAD corpus more accessible by translating it. Their study showed some of the challenges of implementing QA approaches. between different languages that have different linguistic features. Ali et al. [9] reviewed multilingual relation extraction and discussed challenges associated with uneven resource availability and multilingual representation.

These studies motivate the development of language-aware retrieval and evaluation approaches for regional-language QA

systems, particularly for languages with limited resources and distinct linguistic structures.

B. Semantic Retrieval and Multilingual Representation

Dense retrieval uses embeddings to encode queries and documents and matches them by semantic relevance. Hambarde and Proença [10] examined sparse, dense, and hybrid information retrieval approaches and emphasized the importance of retrieval and ranking quality.

Multilingual embedding models enable representations from multiple languages to be mapped into a shared semantic space, supporting cross-lingual retrieval. DesiQuery uses BAAI/bge-m3 embeddings for multilingual semantic matching, which is stored in the local vector repository called ChromaDB.

However, retrieving a relevant document chunk alone does not guarantee that the generated response will remain faithful to the retrieved evidence. Therefore, semantic retrieval must be combined with evidence grounding and response verification to improve the reliability of multilingual QA.

C. LLM Hallucination and Response Reliability

Hallucination and unsupported generation remain significant limitations of LLM-based QA systems. Hu et al. [11] explored how to incorporate outside knowledge into VQA and highlighted the importance of grounded information. A multi-perspective consistency method to identify hallucinations in LLM was introduced by Kong et al. [12]. Behmanesh et al. [13] investigated confidence-based answer selection for knowledge-based question answering.

These studies demonstrate that improving the language model alone is not sufficient to achieve reliable QA. Retrieval quality, evidence verification, and grounding of generated answers are also important. The proposed DesiQuery system addresses these challenges by restricting response generation to retrieved document evidence and retaining source and page metadata for traceability.

D. Local and Resource-Constrained LLM Deployment

Local deployment introduces additional constraints related to model size, inference time, memory requirements, and available computational resources. Cerqueira [14] conducted a quantitative evaluation of quantized local LLMs for RAG under resource-constrained settings, considering answer quality, retrieval performance, and runtime behavior. Choi et al. [15] investigated the generation of quality test questions and answers using quantized low-rank adapters in LLMs. Liao et al. [16] explored cross-attention reprogramming for automatic speech recognition by connecting discrete speech units with pretrained language models.

These studies illustrate the importance of efficient deployment and model adaptation under resource constraints. In contrast, DesiQuery uses BAAI/bge-m3 embeddings, local ChromaDB retrieval, and Llama 3 through Ollama without requiring fine-tuning or cloud-hosted generation. This architecture enables the document knowledge base to be updated without retraining the language model.

TABLE I
COMPARATIVE ANALYSIS OF DESIQUERY AND EXISTING WORKS

Study	Main Focus	Retrieval Knowledge Method	Language Domain	Local Generation	Sourceability	Traceability
Khan al. [1]	et Urdu Question Answering	Lexical and semantic retrieval	Urdu	Not the primary focus	Limited	
Amal al. [2]	et Explainable educational QA	Multimodal RAG and token grounding	Educational	Not the primary focus	Grounding-based	
Hindi al. [3]	et RAG in legal technology	Retrieval-based generation	Legal domain	Not the primary focus	Interpretability-focused	
Xu al. [4]	et Factuality-aware QA	Knowledge graph enhanced retrieval	Traditional folklore	Not the primary focus	Knowledge-graph based	
Saha al. [5]	et Retrieval improvement	Inverted question matching	General QA	Not the primary focus	Not the primary focus	
Cerqueira et al. [14]	et Local RAG deployment	Retrieval with local LLM	Resource-constrained setting	Yes	Not the primary focus	
DesiQuery	Regional-language document QA	BGE-M3 ChromaDB	+ Eight Indian languages	Llama 3 + Ollama	Document, page, chunk and similarity	

The existing research has paved the way for RAG, Multilingual QA, Semantic Retrieval, Hallucination Detection, and Local LLM Deployment, but they are studied in isolation or in limited domains. DesiQuery integrates multilingual BAAI/bge-m3 embeddings, retrieval with ChromaDB, Llama 3 (with Ollama) and source-level information for document QA in regional language. System evaluation is conducted over eight Indian languages for both supported and unsupported queries, based on language consistency, language grounding, source traceability, response completeness and latency.

III. PROPOSED SYSTEM

DesiQuery is a multilingual, document-grounded question-answering framework designed for local deployment. It provides answers in eight Indian languages while ensuring that responses are generated only from the information contained in the selected documents. Users can select pre-loaded documents or upload PDF, TXT, and DOCX files and submit queries in any supported language. The system architecture is divided into modular stages covering document processing, language handling, semantic retrieval, response generation, and source attribution.

The developed system consists of six major modules: (1) Document Collection and Preprocessing, (2) Language Processing and Normalization, (3) Document Chunking and BGE-M3 Embedding, (4) ChromaDB and Semantic Retrieval, (5) RAG with Ollama and a Local LLM, and (6) Source Document and Page Number Tracking. The modular design allows individual components to be replaced, inspected, and diagnosed independently without duplicating their responsibilities.

A. Document Collection and Preprocessing

The system accepts PDF, TXT, and DOCX documents that may either be pre-loaded or uploaded by the user. During ingestion, text is extracted from the selected documents and associated with the corresponding document name and page number whenever available. Documents containing insufficient extractable text, such as image-only PDFs, are rejected during the ingestion stage. The preserved document and page metadata are subsequently used during retrieval and source attribution.

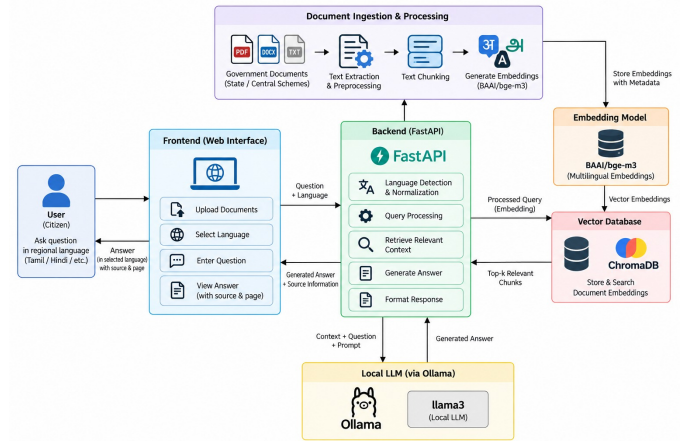


Fig. 1. System Architecture

B. Language Processing and Normalization

DesiQuery supports eight different Indian Regional languages. The user selects the language before submitting a query, and the selected language is used throughout the query processing and answer generation stages. The generated response is also expected to remain consistent with the selected language. Language consistency is subsequently validated during system evaluation.

C. Document Chunking and BGE-M3 Embeddings

The extracted document text is segmented into smaller chunks to enable fine-grained semantic retrieval. Each chunk retains its associated document identifier, chunk identifier, and page information whenever available. The BAAI/bge-m3 embedding model is used to generate multilingual semantic representations of both document chunks and user queries. The resulting embeddings are stored along with their metadata for subsequent retrieval.

D. ChromaDB and Semantic Retrieval

ChromaDB is used as the local vector database for storing document embeddings and associated metadata. When a user submits a query, its semantic representation is generated using the same embedding model and compared with the indexed document representations. The most relevant chunks are retrieved and ranked according to their semantic similarity. The retrieved evidence and associated metadata are retained for context construction and source attribution.

E. RAG and Local LLM Generation

The retrieved evidence is supplied as context to Llama 3 through Ollama for evidence-constrained generation. The generation process is constrained to the retrieved document evidence and the selected output language. If the required information is not present in the retrieved evidence, the system indicates that sufficient information is not available rather than generating unsupported content.

Context selection is performed before generation to retain the most relevant evidence while controlling the amount of information provided to the local language model. This method is used to minimize unsupported generation by selecting the most pertinent retrieved evidence for the generation of a response.

F. Source Document and Page Number Tracking

The system maintains source information associated with the retrieved chunks, including the source document, page number, chunk identifier, and retrieval similarity score when available. The generated answer is returned together with its supporting source information, allowing users to inspect the evidence used for answer generation. The similarity score is presented as a retrieval indicator and is not interpreted as a probability or confidence score.

G. Overall Processing Pipeline

The overall processing pipeline begins when the user selects or uploads documents through the Document Upload interface and initiates text extraction. The extracted content is processed, normalized, divided into chunks, and converted into BGE-M3 embeddings. The embeddings are then indexed in ChromaDB for semantic retrieval. When a query is submitted, its embedding is generated and compared with the indexed document representations. Relevant evidence is selected and supplied to Llama 3 through Ollama for grounded answer generation. Finally, the answer is returned together with the corresponding source attribution.

The complete processing flow is represented as:

Document Upload → Text Extraction → Language Processing
 → Chunking → BGE-M3 Embedding → ChromaDB
 → Semantic Retrieval → Context Selection
 → Llama 3/Ollama → Answer + Source Attribution

This modular architecture separates document preparation, semantic retrieval, response generation, and evidence traceability, enabling pipeline-level monitoring and diagnosis of individual processing stages.

IV. METHODOLOGY

The document grounded multilingual QA system, Desi Query, is a retrieval + generation based approach that employs locally hosted LLM.

Desi Query:

- ingests content in eight languages (English, Tamil, Hindi, Telugu, Malayalam, Marathi, Bengali, Kannada),
- normalizes and chunks it,
- applies multilingual embedding and,
- performs vector storage and semantic retrieval,
- selects context from the retrieved data,
- generates answer using locally hosted LLM, validates answer to language of concern, and,
- attributes source to generated answer.

A. Document processing chunking:

Desi Query accepts PDF, TXT, and DOCX documents. Supported PDF files are retained in page granularity if page number information can be retrieved reliably from the file. Text-only documents with very limited content are ignored. Content collection D is represented as

$$D = \{d_1, d_2, \dots, d_n\}$$

Document chunks are stored for future retrieval,

$$d_i \rightarrow \{ci_1, ci_2, \dots, ci_m\}$$

Chunk ID, Page Number are preserved to support source attribution with output chunks.

B. Embedding semantic retrieval:

Both document chunks and queries are transformed to vectors using BAAI/bge-m3,

$$e_i = f_{\text{BGE-M3}}(c_i), \quad e_q = f_{\text{BGE-M3}}(q)$$

Vectors are stored in ChromaDB, and chunks similar to query are identified by cosine similarity:

$$\text{sim}(q, c_i) = \frac{e_q \cdot e_i}{\|e_q\| \|e_i\|}$$

Ranking based on this measurement yields the evidence set. Sentence based filtering to remove redundant information is performed to improve quality of the content before text generation.

C. Retrieval-augmented generation:

Contextually selected evidence is combined with query and fed to Llama 3 via Ollama:

$$A = G(q, C'_q)$$

where A is answer, q is question, C'_q is filtered context, and G is the locally hosted LLM. Llama is instructed to respond in the language of interest, use only provided evidence, avoid unsupported statements and, communicate when information cannot be found in documents.

D. Validating attributing output:

The output is validated for content and language, and context source information, including document name, page number, chunk ID, and similarity score, is provided with the answer. In particular, similarity scores are indicators of retrieval relevance, rather than probabilities of answer correctness. When evidence for answering a question is not available, the answer is reported as "answer not available". In addition, generation truncation flag is observed and, in case of a truncated answer, a new prompt with a larger maximum output token count is sent to Llama.

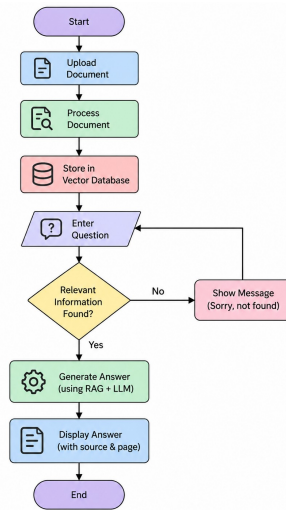


Fig. 2. Algorithm Workflow

V. EXPERIMENTAL SETUP AND RESULTS

A. Experimental Setup

The DesiQuery system was evaluated to assess its performance in document-grounded multilingual question answering. The evaluation focused on language consistency, evidence grounding, source traceability, response completeness, and response latency.

A total of 24 test cases were constructed using three question categories: procedure-based questions, application/document-related questions, and unsupported questions. Each category contained one test case for each of the eight supported languages. Thus, each language was evaluated using three test cases.

The evaluation was performed through the FastAPI query interface using the complete processing pipeline, including query embedding, semantic retrieval with ChromaDB, context selection, and local Llama 3 generation through Ollama. Retrieval metadata was also verified to ensure that generated responses could be traced to the retrieved document evidence.

TABLE II
MULTILINGUAL EVALUATION TEST SET

Category	EN	TA	HI	TE	KN	ML	BN	MR	Total
Procedure	1	1	1	1	1	1	1	1	8
Application/Documentation	1	1	1	1	1	1	1	1	8
Unsupported	1	1	1	1	1	1	1	1	8
Total	3	3	3	3	3	3	3	3	24

B. Evaluation Criteria

Five criteria were considered during evaluation. Language consistency checks whether the response follows the selected output language. Grounding assesses whether statements, numerical details, and procedural instructions in the response can be traced to the selected document content. Source traceability verifies the consistency of document, chunk, and page metadata when available. Response completeness checks

whether the answer terminates naturally within the configured generation limits. Response latency indicates the time taken to process a query and generate the corresponding answer.

The evaluation was implemented using automated validation together with diagnostic information produced by the backend. Grounding validation specifically measures support from the retrieved evidence and is not treated as a general measure of factual accuracy.

C. Results and Analysis

The evaluation produced 19 PASS, 5 WARNING, and 0 FAIL results across the 24 test cases. Language validation, grounding validation, and source traceability achieved successful validation for all evaluated cases. The warning cases were associated primarily with response completion under the current local generation configuration.

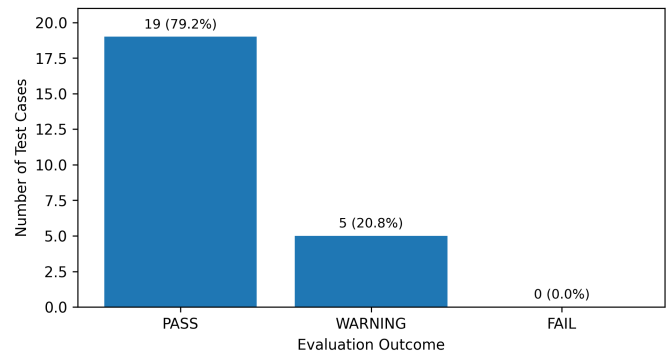


Fig. 3. Overall Evaluation Results

The language-wise results are presented in Table III. English, Tamil, Hindi, Bengali, and Marathi produced PASS results for all three test cases. Kannada produced two PASS and one WARNING, while Telugu and Malayalam produced one PASS and two WARNING results each. These warning cases indicate that response completeness can vary for some regional-language outputs under the current local LLM configuration. They should be interpreted within the scope of the evaluated test set and not as a general ranking of language performance.

TABLE III
LANGUAGE-WISE EVALUATION RESULTS

Language	PASS	WARNING	FAIL	Total
English	3	0	0	3
Tamil	3	0	0	3
Hindi	3	0	0	3
Telugu	1	2	0	3
Kannada	2	1	0	3
Malayalam	1	2	0	3
Bengali	3	0	0	3
Marathi	3	0	0	3
Total	19	5	0	24

The retrieval diagnostics showed that semantic retrieval generally required substantially less time than local LLM

generation. Most observed retrieval operations were completed within approximately 0.24–0.63 s, with one measured retrieval time of 8.10 s. The retrieved chunks were retained for inspection, while context selection reduced unnecessary or weakly relevant content before generation. A formal Recall@K or Mean Reciprocal Rank value is not reported because the current evaluation set does not contain explicit relevance labels required for these metrics.

The latency measurements further showed that local LLM inference was the dominant contributor to end-to-end response time. The observed first-generation inference times in the available diagnostic measurements ranged from approximately 52.63 to 84.58 s, while the average total response time across the current 24-test evaluation was approximately 102 s. These measurements are specific to the local hardware and inference configuration and should not be considered universal performance values. The results suggest that the local LLM inference constitutes the larger portion of the end-to-end latency in the current implementation, and semantic retrieval typically takes much less time.

D. Reliability and Discussion

The system can handle unsupported queries, incomplete/corrupted documents, generation failures, invalid metadata, and incomplete responses. When the information cannot be found in the selected documents, the system does not fall back to unsupported model knowledge, but reports the information is not available. The evaluation results demonstrate language fidelity, grounding, and source traceability across the supported eight languages, although additional work is needed to enhance response completeness for Telugu, Kannada, and Malayalam.

The main bottleneck in performance is LLM inference within the local inference engine. This evaluation used only 24 test cases and performance is hardware-dependent, so these results reflect the system's performance in the tested environment, but not generally for the full set of regional-language queries and document types.

VI. CONCLUSION AND FUTURE SCOPE

DesiQuery is a locally deployed multilingual RAG framework for Document Grounded Question Answering across 8 Indian languages, built using BAAI/bge-m3, ChromaDB, Ollama model of Llama 3, along with source, page, chunk and similarity traceability. The evaluation of 24 test cases resulted in 19 PASS, 5 WARNING and 0 FAIL cases, with successful language, grounding and source-traceability validation for the test set. The results validate the proposed workflow and identify the local LLM inference latency and limitations in response completion as areas for future enhancements.

Future work will focus on expanding the multilingual evaluation dataset, creating formally annotated retrieval benchmarks, and conducting ablation studies for retrieval depth and context filtering. Further improvements will investigate optimized or quantized local models to reduce inference latency, improved processing of scanned PDFs, tables and complex

document structures, and language-specific evaluation and optimization for regional-language responses. These extensions can improve the retrieval reliability, multilingual robustness, and computational efficiency of DesiQuery.

REFERENCES

- [1] T. F. Khan, M. Arslan, F. Alturise, A. H. Almutairi, and M. T. Raza, "Uqa-rag: Enhancing urdu question-answer retrieval with semantic similarity and llms," *IEEE Access*, vol. 14, pp. 2868–2882, 2026.
- [2] S. Amal, A. R. Niranjana, and M. G. Thushara, "Edurag: A multimodal explainable rag system with deterministic token grounding," *IEEE Access*, vol. 14, pp. 90 849–90 859, 2026.
- [3] M. Hindi, L. Mohammed, O. Maaz, and A. Alwarafy, "Enhancing the precision and interpretability of retrieval-augmented generation (rag) in legal technology: A survey," *IEEE Access*, vol. 13, pp. 46 171–46 189, 2025.
- [4] J. Xu, H. Zhang, H. Zhang, J. Lu, and G. Xiao, "Chattf: A knowledge graph-enhanced intelligent q&a system for mitigating factuality hallucinations in traditional folklore," *IEEE Access*, vol. 12, pp. 162 638–162 650, 2024.
- [5] B. Saha, U. Saha, and M. Zubair Malik, "Quim-rag: Advancing retrieval-augmented generation with inverted question matching for enhanced qa performance," *IEEE Access*, vol. 12, pp. 185 401–185 410, 2024.
- [6] A. Onan, A. H. Nasution, T. Celikten, and P. Cetin, "Comparing chatgpt, gemini, and emerging llms in low-resource educational settings: Reasoning quality, consistency, and explainability," *IEEE Access*, vol. 14, pp. 32 807–32 838, 2026.
- [7] A. Perevalov, A. Gashkov, M. Eltsova, and A. Both, "Sparql query candidate filtering for improving the quality of multilingual question answering over knowledge graphs using language models," *Journal of Web Engineering*, vol. 24, no. 4, pp. 563–592, 2025.
- [8] A. Vijeevaraj Ann Sinthusha, E. Y. A. Charles, and R. Weerasinghe, "Machine reading comprehension for the tamil language with translated squad," *IEEE Access*, vol. 13, pp. 13 312–13 328, 2025.
- [9] M. Ali, R. Speck, H. M. Zahera, M. Saleem, D. Moussallem, and A.-C. Ngonga Ngomo, "Multilingual relation extraction: A survey," *IEEE Access*, vol. 13, pp. 151 907–151 933, 2025.
- [10] K. A. Hambarde and H. Proença, "Information retrieval: Recent advances and beyond," *IEEE Access*, vol. 11, pp. 76 581–76 604, 2023.
- [11] Z. Hu, P. Yang, F. Liu, Y. Meng, and X. Liu, "Prompting large language models with knowledge-injection for knowledge-based visual question answering," *Big Data Mining and Analytics*, vol. 7, no. 3, pp. 843–857, 2024.
- [12] L. Kong, X. Zhong, J. Chen, H. Fu, and Y. Wang, "Multi-perspective consistency checking for large language model hallucination detection: A black-box zero-resource approach," *Frontiers of Information Technology & Electronic Engineering*, vol. 26, no. 11, pp. 2298–2309, 2025.
- [13] S. Behmanesh, A. Talebpour, M. Shamsfard, and M. Mahdi Jafari, "A novel open-domain question answering system on curated and extracted knowledge bases with consideration of confidence scores in existing triples," *IEEE Access*, vol. 12, pp. 160 741–160 760, 2024.
- [14] M. L. Lima de Oliveira and J. de Jesus Fiais Cerqueira, "Resource-constrained evaluation of quantized local llms for retrieval-augmented generation on a reduced-corpus setting," *IEEE Access*, vol. 14, pp. 70 402–70 422, 2026.
- [15] J. Choi, S. Hong, S. Hong, J. Park, and E.-S. Jung, "Toward generating quality test questions and answers using quantized low-rank adapters in llms," *IEEE Access*, vol. 13, pp. 87 793–87 809, 2025.
- [16] P.-J. Liao, H.-Y. Lee, and H.-M. Wang, "Cross-attention reprogramming for asr: Bridging discrete speech units and pretrained language models," *IEEE Access*, vol. 14, pp. 662–678, 2026.

Dr Vani

T043_PAPER .pdf

 Rajalakshmi Engineering College

Document Details

Submission ID

trn:oid::3618:153500330

Submission Date

Oct 2, 2026, 1:07 PM GMT+5:30

Download Date

Oct 2, 2026, 1:14 PM GMT+5:30

File Name

T043_PAPER .pdf

File Size

617.8 KB

6 Pages

4,268 Words

25,917 Characters

2% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Exclusions

- 6 Excluded Matches

Match Groups

-  **5 Not Cited or Quoted 1%**
Matches with neither in-text citation nor quotation marks
-  **3 Missing Quotations 1%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 1%  Internet sources
- 1%  Publications
- 0%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

-  **5 Not Cited or Quoted** 1%
Matches with neither in-text citation nor quotation marks
-  **3 Missing Quotations** 1%
Matches that are still very similar to source material
-  **0 Missing Citation** 0%
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted** 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 1%  Internet sources
- 1%  Publications
- 0%  Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

- 1** Publication
Pei-Jun Liao, Hung-Yi Lee, Hsin-Min Wang. "Cross-Attention Reprogramming for A... <1%
- 2** Internet
www.mdpi.com <1%
- 3** Publication
Divija Amaram, Lu Gao, Gowtham Reddy Gudla, Tejaswini Sanjay Katale. "Develop... <1%
- 4** Publication
Paramita Ray, Saptarshe Banerjee. "EKAMG: Enhanced Knowledge-Augmented M... <1%
- 5** Internet
mdpi-res.com <1%
- 6** Internet
pages.milaap.org <1%
- 7** Internet
rsisinternational.org <1%

DesiQuery: An Evidence-Grounded Multilingual Framework for Regional Language Question Answering

Chettiyar Vani Vivekanand

*Department of Computer
Science and Engineering*

Rajalakshmi Engineering College

Chennai, India

chettiyarvanivivekanand@rajalakshmi.edu.in

Kabilesh P

*Department of Computer
Science and Engineering*

Rajalakshmi Engineering College

Chennai, India

2307011333@rajalakshmi.edu.in

Manishaa G

*Department of Computer
Science and Engineering*

Rajalakshmi Engineering College

Chennai, India

230701176@rajalakshmi.edu.in

Manikandan M

*Department of Computer
Science and Engineering*

Rajalakshmi Engineering College

Chennai, India

230701174@rajalakshmi.edu.in

Madhumitha P

*Department of Computer
Science and Engineering*

Rajalakshmi Engineering College

Chennai, India

230701169@rajalakshmi.edu.in

Abstract—Access to reliable digital information in Indian regional languages remains challenging due to linguistic diversity, limited language-specific resources, and the tendency of large language models (LLMs) to generate unsupported information when domain-specific evidence is unavailable. The developed DesiQuery framework allows users to select or upload documents and ask questions in eight Indian languages, where BAAI/bge-m3 is used to identify semantically relevant document chunks in ChromaDB and Llama 3 generates responses through Ollama using the retrieved evidence. To improve reliability and transparency, the system restricts answer generation to retrieved evidence and provides source-document, page-level, chunk-level, and similarity information when available. Queries for which relevant information cannot be retrieved are explicitly identified rather than being answered using unsupported model knowledge. The system was evaluated using a multilingual test suite covering supported and unsupported information-seeking queries across the eight target languages, with evaluation focused on language consistency, grounding, source traceability, response completeness, and latency. The results demonstrate the feasibility of locally deployed, source-traceable RAG for regional-language document question answering while also identifying response-generation limitations for some longer multilingual outputs. The proposed framework provides a privacy-oriented and extensible approach for accessing document-based information in Indian regional languages without dependence on cloud-based LLM APIs.

Index Terms—Regional Language Processing, Question Answering, Retrieval-Augmented Generation, Large Language Models, IndicNLP, ChromaDB, Semantic Retrieval, Ollama, Natural Language Processing.

I. INTRODUCTION

An era of rapidly expanding digital information has increased the use of Question Answering (QA) systems for retrieving information from large document collections. However, reliable document-based QA remains challenging for Indian regional languages due to differences in vocabulary,

scripts, morphology, and syntax. The challenge becomes more significant when answers must be derived from specific documents such as government, administrative, or domain-specific resources.

Large Language Models (LLMs) exhibit great capabilities for language understanding and generation; however, these models may produce information which is not part of the documents provided. The challenge of extracting the relevant external sources and then feeding the LLM with these sources can be addressed by employing Retrieval-Augmented Generation (RAG). Multilingual QA requires an appropriate semantic representation across the languages for efficient retrieval and generation of grounded response. In some instances, local deployment may be necessary because of the sensitivity of the information in the documents; local execution of the retrieval and language model pipeline ensures greater control over these issues. To address these challenges, we present DesiQuery, a locally deployed multilingual document-grounded QA system based on Retrieval-Augmented Generation.

This proposed system combines multilingual semantic representations using BAAI/bge-m3, ChromaDB for vector storage and semantic retrieval, and Llama 3 with Ollama for local response generation. The system supports eight Indian languages and constrains generated responses to the retrieved document evidence. It also maintains source-document, page, chunk, and similarity metadata, allowing users to inspect the evidence associated with each generated answer. If the required information is not available in the selected documents, the system indicates that the information is unavailable rather than generating an unsupported response.

The main contributions of this work are:

- 1) **Multilingual document-grounded QA:** A multilingual question-answering system is developed for document-

grounded question answering across eight Indian languages, enabling semantic retrieval and response generation in regional languages.

- 2) **Local QA with source traceability:** A streamlined local QA workflow is developed using BAAI/bge-m3, ChromaDB, and Llama 3 through Ollama. The system incorporates source document, page, chunk, and similarity metadata to provide evidence traceability for generated answers.
- 3) **Evidence-grounded evaluation:** The complete pipeline is evaluated using both supported and unsupported queries across the supported languages, considering language consistency, grounding, source traceability, response completeness, and response latency.

II. RELATED WORKS

Recent development of LLMs has brought enhancement in the capability of QA system for natural language understanding and knowledge-based response generation. But when the information is in a specialized document, the responses created by LLM can be hard to verify. RAG tackles this by extracting relevant data prior to generating a response. Khan et al. [1] suggested UQA-RAG for Urdu QA system that involved lexical and semantic retrieval, and Amal et al. [2] introduced another deterministic token-based grounding-based system named EduRAG for Urdu QA. Hindi et al. [3] discussed the use of RAG in legal domain, Xu et al. [4] introduced ChatTf with knowledge graphs for factuality and Saha et al. [5] presented QuIM-RAG by inverse question matching. Retrieval and grounding play an important role in these studies, but they concentrate primarily on single retrieval methods or retrieval areas, and not on a multi-lingual regional-language QA, local inference, and source-level traceability.

A. Multilingual and Regional-Language Question Answering

Multiple languages in a QA system pose difficulties. This is because of the differences in word usage, grammar and writing patterns. In the context Onan et al. [6] present a collection of low-resource educational applications. Reviewed the performance of cross-modal LLM QA, focusing on quality Logical and consistent answers and explainability. The work of Perevalov et al. [7] addressed the question of QA on multilingual knowledge graphs and explored the use of language models to filter candidate queries. Language-specific regional-language QA has also been investigated. Sinthusha et al. [8] were concerned with Tamil machine reading. This involves improving comprehension by making a dataset from the SQuAD corpus more accessible by translating it. Their study showed some of the challenges of implementing QA approaches. between different languages that have different linguistic features. Ali et al. [9] reviewed multilingual relation extraction and discussed challenges associated with uneven resource availability and multilingual representation.

These studies motivate the development of language-aware retrieval and evaluation approaches for regional-language QA

systems, particularly for languages with limited resources and distinct linguistic structures.

B. Semantic Retrieval and Multilingual Representation

Dense retrieval uses embeddings to encode queries and documents and matches them by semantic relevance. Hambarde and Proença [10] examined sparse, dense, and hybrid information retrieval approaches and emphasized the importance of retrieval and ranking quality.

Multilingual embedding models enable representations from multiple languages to be mapped into a shared semantic space, supporting cross-lingual retrieval. DesiQuery uses BAAI/bge-m3 embeddings for multilingual semantic matching, which is stored in the local vector repository called ChromaDB.

However, retrieving a relevant document chunk alone does not guarantee that the generated response will remain faithful to the retrieved evidence. Therefore, semantic retrieval must be combined with evidence grounding and response verification to improve the reliability of multilingual QA.

C. LLM Hallucination and Response Reliability

Hallucination and unsupported generation remain significant limitations of LLM-based QA systems. Hu et al. [11] explored how to incorporate outside knowledge into VQA and highlighted the importance of grounded information. A multi-perspective consistency method to identify hallucinations in LLM was introduced by Kong et al. [12]. Behmanesh et al. [13] investigated confidence-based answer selection for knowledge-based question answering.

These studies demonstrate that improving the language model alone is not sufficient to achieve reliable QA. Retrieval quality, evidence verification, and grounding of generated answers are also important. The proposed DesiQuery system addresses these challenges by restricting response generation to retrieved document evidence and retaining source and page metadata for traceability.

D. Local and Resource-Constrained LLM Deployment

Local deployment introduces additional constraints related to model size, inference time, memory requirements, and available computational resources. Cerqueira [14] conducted a quantitative evaluation of quantized local LLMs for RAG under resource-constrained settings, considering answer quality, retrieval performance, and runtime behavior. Choi et al. [15] investigated the generation of quality test questions and answers using quantized low-rank adapters in LLMs. Liao et al. [16] explored cross-attention reprogramming for automatic speech recognition by connecting discrete speech units with pretrained language models.

These studies illustrate the importance of efficient deployment and model adaptation under resource constraints. In contrast, DesiQuery uses BAAI/bge-m3 embeddings, local ChromaDB retrieval, and Llama 3 through Ollama without requiring fine-tuning or cloud-hosted generation. This architecture enables the document knowledge base to be updated without retraining the language model.

TABLE I
COMPARATIVE ANALYSIS OF DESIQUERY AND EXISTING WORKS

Study	Main Focus	Retrieval Knowledge Method	Language Domain	Local Generation	Sourceability	Traceability
Khan al. [1]	et Urdu Question Answering	Lexical and semantic retrieval	Urdu	Not the primary focus	Limited	
Amal al. [2]	et Explainable educational QA	Multimodal RAG and token grounding	Educational	Not the primary focus	Grounding-based	
Hindi al. [3]	et RAG in legal technology	Retrieval-based generation	Legal domain	Not the primary focus	Interpretability-focused	
Xu al. [4]	et Factuality-aware QA	Knowledge graph enhanced retrieval	Traditional folklore	Not the primary focus	Knowledge-graph based	
Saha al. [5]	et Retrieval improvement	Inverted question matching	General QA	Not the primary focus	Not the primary focus	
Cerqueira et al. [14]	et Local RAG deployment	Retrieval with local LLM	Resource-constrained setting	Yes	Not the primary focus	
DesiQuery	Regional-language document QA	BGE-M3 ChromaDB	+ Eight Indian languages	Llama 3 + Ollama	Document, page, chunk and similarity	

The existing research has paved the way for RAG, Multilingual QA, Semantic Retrieval, Hallucination Detection, and Local LLM Deployment, but they are studied in isolation or in limited domains. DesiQuery integrates multilingual BAAI/bge-m3 embeddings, retrieval with ChromaDB, Llama 3 (with Ollama) and source-level information for document QA in regional language. System evaluation is conducted over eight Indian languages for both supported and unsupported queries, based on language consistency, language grounding, source traceability, response completeness and latency.

III. PROPOSED SYSTEM

DesiQuery is a multilingual, document-grounded question-answering framework designed for local deployment. It provides answers in eight Indian languages while ensuring that responses are generated only from the information contained in the selected documents. Users can select pre-loaded documents or upload PDF, TXT, and DOCX files and submit queries in any supported language. The system architecture is divided into modular stages covering document processing, language handling, semantic retrieval, response generation, and source attribution.

The developed system consists of six major modules: (1) Document Collection and Preprocessing, (2) Language Processing and Normalization, (3) Document Chunking and BGE-M3 Embedding, (4) ChromaDB and Semantic Retrieval, (5) RAG with Ollama and a Local LLM, and (6) Source Document and Page Number Tracking. The modular design allows individual components to be replaced, inspected, and diagnosed independently without duplicating their responsibilities.

A. Document Collection and Preprocessing

The system accepts PDF, TXT, and DOCX documents that may either be pre-loaded or uploaded by the user. During ingestion, text is extracted from the selected documents and associated with the corresponding document name and page number whenever available. Documents containing insufficient extractable text, such as image-only PDFs, are rejected during the ingestion stage. The preserved document and page metadata are subsequently used during retrieval and source attribution.

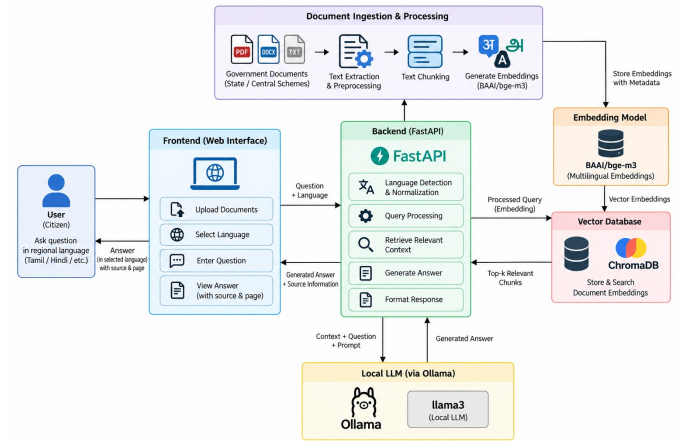


Fig. 1. System Architecture

B. Language Processing and Normalization

DesiQuery supports eight different Indian Regional languages. The user selects the language before submitting a query, and the selected language is used throughout the query processing and answer generation stages. The generated response is also expected to remain consistent with the selected language. Language consistency is subsequently validated during system evaluation.

C. Document Chunking and BGE-M3 Embeddings

The extracted document text is segmented into smaller chunks to enable fine-grained semantic retrieval. Each chunk retains its associated document identifier, chunk identifier, and page information whenever available. The BAAI/bge-m3 embedding model is used to generate multilingual semantic representations of both document chunks and user queries. The resulting embeddings are stored along with their metadata for subsequent retrieval.

D. ChromaDB and Semantic Retrieval

ChromaDB is used as the local vector database for storing document embeddings and associated metadata. When a user submits a query, its semantic representation is generated using the same embedding model and compared with the indexed document representations. The most relevant chunks are retrieved and ranked according to their semantic similarity. The retrieved evidence and associated metadata are retained for context construction and source attribution.

E. RAG and Local LLM Generation

The retrieved evidence is supplied as context to Llama 3 through Ollama for evidence-constrained generation. The generation process is constrained to the retrieved document evidence and the selected output language. If the required information is not present in the retrieved evidence, the system indicates that sufficient information is not available rather than generating unsupported content.

Context selection is performed before generation to retain the most relevant evidence while controlling the amount of information provided to the local language model. This method is used to minimize unsupported generation by selecting the most pertinent retrieved evidence for the generation of a response.

F. Source Document and Page Number Tracking

The system maintains source information associated with the retrieved chunks, including the source document, page number, chunk identifier, and retrieval similarity score when available. The generated answer is returned together with its supporting source information, allowing users to inspect the evidence used for answer generation. The similarity score is presented as a retrieval indicator and is not interpreted as a probability or confidence score.

G. Overall Processing Pipeline

The overall processing pipeline begins when the user selects or uploads documents through the Document Upload interface and initiates text extraction. The extracted content is processed, normalized, divided into chunks, and converted into BGE-M3 embeddings. The embeddings are then indexed in ChromaDB for semantic retrieval. When a query is submitted, its embedding is generated and compared with the indexed document representations. Relevant evidence is selected and supplied to Llama 3 through Ollama for grounded answer generation. Finally, the answer is returned together with the corresponding source attribution.

The complete processing flow is represented as:

Document Upload → Text Extraction → Language Processing
 → Chunking → BGE-M3 Embedding → ChromaDB
 → Semantic Retrieval → Context Selection
 → Llama 3/Ollama → Answer + Source Attribution

This modular architecture separates document preparation, semantic retrieval, response generation, and evidence traceability, enabling pipeline-level monitoring and diagnosis of individual processing stages.

IV. METHODOLOGY

The document grounded multilingual QA system, Desi Query, is a retrieval + generation based approach that employs locally hosted LLM.

Desi Query:

- ingests content in eight languages (English, Tamil, Hindi, Telugu, Malayalam, Marathi, Bengali, Kannada),
- normalizes and chunks it,
- applies multilingual embedding and,
- performs vector storage and semantic retrieval,
- selects context from the retrieved data,
- generates answer using locally hosted LLM, validates answer to language of concern, and,
- attributes source to generated answer.

A. Document processing chunking:

Desi Query accepts PDF, TXT, and DOCX documents. Supported PDF files are retained in page granularity if page number information can be retrieved reliably from the file. Text-only documents with very limited content are ignored. Content collection D is represented as

$$D = \{d_1, d_2, \dots, d_n\}$$

Document chunks are stored for future retrieval,

$$d_i \rightarrow \{ci_1, ci_2, \dots, ci_m\}$$

Chunk ID, Page Number are preserved to support source attribution with output chunks.

B. Embedding semantic retrieval:

Both document chunks and queries are transformed to vectors using BAAI/bge-m3,

$$e_i = f_{\text{BGE-M3}}(c_i), \quad e_q = f_{\text{BGE-M3}}(q)$$

Vectors are stored in ChromaDB, and chunks similar to query are identified by cosine similarity:

$$\text{sim}(q, c_i) = \frac{e_q \cdot e_i}{\|e_q\| \|e_i\|}$$

Ranking based on this measurement yields the evidence set. Sentence based filtering to remove redundant information is performed to improve quality of the content before text generation.

C. Retrieval-augmented generation:

Contextually selected evidence is combined with query and fed to Llama 3 via Ollama:

$$A = G(q, C'_q)$$

where A is answer, q is question, C'_q is filtered context, and G is the locally hosted LLM. Llama is instructed to respond in the language of interest, use only provided evidence, avoid unsupported statements and, communicate when information cannot be found in documents.

D. Validating attributing output:

The output is validated for content and language, and context source information, including document name, page number, chunk ID, and similarity score, is provided with the answer. In particular, similarity scores are indicators of retrieval relevance, rather than probabilities of answer correctness. When evidence for answering a question is not available, the answer is reported as "answer not available". In addition, generation truncation flag is observed and, in case of a truncated answer, a new prompt with a larger maximum output token count is sent to Llama.

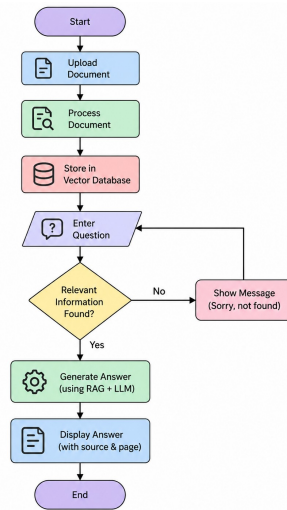


Fig. 2. Algorithm Workflow

V. EXPERIMENTAL SETUP AND RESULTS

A. Experimental Setup

The DesiQuery system was evaluated to assess its performance in document-grounded multilingual question answering. The evaluation focused on language consistency, evidence grounding, source traceability, response completeness, and response latency.

A total of 24 test cases were constructed using three question categories: procedure-based questions, application/document-related questions, and unsupported questions. Each category contained one test case for each of the eight supported languages. Thus, each language was evaluated using three test cases.

The evaluation was performed through the FastAPI query interface using the complete processing pipeline, including query embedding, semantic retrieval with ChromaDB, context selection, and local Llama 3 generation through Ollama. Retrieval metadata was also verified to ensure that generated responses could be traced to the retrieved document evidence.

TABLE II
MULTILINGUAL EVALUATION TEST SET

Category	EN	TA	HI	TE	KN	ML	BN	MR	Total
Procedure	1	1	1	1	1	1	1	1	8
Application/Documentation	1	1	1	1	1	1	1	1	8
Unsupported	1	1	1	1	1	1	1	1	8
Total	3	3	3	3	3	3	3	3	24

B. Evaluation Criteria

Five criteria were considered during evaluation. Language consistency checks whether the response follows the selected output language. Grounding assesses whether statements, numerical details, and procedural instructions in the response can be traced to the selected document content. Source traceability verifies the consistency of document, chunk, and page metadata when available. Response completeness checks

whether the answer terminates naturally within the configured generation limits. Response latency indicates the time taken to process a query and generate the corresponding answer.

The evaluation was implemented using automated validation together with diagnostic information produced by the backend. Grounding validation specifically measures support from the retrieved evidence and is not treated as a general measure of factual accuracy.

C. Results and Analysis

The evaluation produced 19 PASS, 5 WARNING, and 0 FAIL results across the 24 test cases. Language validation, grounding validation, and source traceability achieved successful validation for all evaluated cases. The warning cases were associated primarily with response completion under the current local generation configuration.

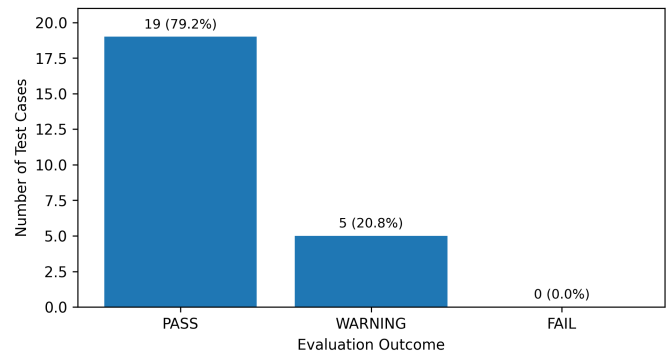


Fig. 3. Overall Evaluation Results

The language-wise results are presented in Table III. English, Tamil, Hindi, Bengali, and Marathi produced PASS results for all three test cases. Kannada produced two PASS and one WARNING, while Telugu and Malayalam produced one PASS and two WARNING results each. These warning cases indicate that response completeness can vary for some regional-language outputs under the current local LLM configuration. They should be interpreted within the scope of the evaluated test set and not as a general ranking of language performance.

TABLE III
LANGUAGE-WISE EVALUATION RESULTS

Language	PASS	WARNING	FAIL	Total
English	3	0	0	3
Tamil	3	0	0	3
Hindi	3	0	0	3
Telugu	1	2	0	3
Kannada	2	1	0	3
Malayalam	1	2	0	3
Bengali	3	0	0	3
Marathi	3	0	0	3
Total	19	5	0	24

The retrieval diagnostics showed that semantic retrieval generally required substantially less time than local LLM

generation. Most observed retrieval operations were completed within approximately 0.24–0.63 s, with one measured retrieval time of 8.10 s. The retrieved chunks were retained for inspection, while context selection reduced unnecessary or weakly relevant content before generation. A formal Recall@K or Mean Reciprocal Rank value is not reported because the current evaluation set does not contain explicit relevance labels required for these metrics.

The latency measurements further showed that local LLM inference was the dominant contributor to end-to-end response time. The observed first-generation inference times in the available diagnostic measurements ranged from approximately 52.63 to 84.58 s, while the average total response time across the current 24-test evaluation was approximately 102 s. These measurements are specific to the local hardware and inference configuration and should not be considered universal performance values. The results suggest that the local LLM inference constitutes the larger portion of the end-to-end latency in the current implementation, and semantic retrieval typically takes much less time.

D. Reliability and Discussion

The system can handle unsupported queries, incomplete/corrupted documents, generation failures, invalid metadata, and incomplete responses. When the information cannot be found in the selected documents, the system does not fall back to unsupported model knowledge, but reports the information is not available. The evaluation results demonstrate language fidelity, grounding, and source traceability across the supported eight languages, although additional work is needed to enhance response completeness for Telugu, Kannada, and Malayalam.

The main bottleneck in performance is LLM inference within the local inference engine. This evaluation used only 24 test cases and performance is hardware-dependent, so these results reflect the system's performance in the tested environment, but not generally for the full set of regional-language queries and document types.

VI. CONCLUSION AND FUTURE SCOPE

DesiQuery is a locally deployed multilingual RAG framework for Document Grounded Question Answering across 8 Indian languages, built using BAAI/bge-m3, ChromaDB, Ollama model of Llama 3, along with source, page, chunk and similarity traceability. The evaluation of 24 test cases resulted in 19 PASS, 5 WARNING and 0 FAIL cases, with successful language, grounding and source-traceability validation for the test set. The results validate the proposed workflow and identify the local LLM inference latency and limitations in response completion as areas for future enhancements.

Future work will focus on expanding the multilingual evaluation dataset, creating formally annotated retrieval benchmarks, and conducting ablation studies for retrieval depth and context filtering. Further improvements will investigate optimized or quantized local models to reduce inference latency, improved processing of scanned PDFs, tables and complex

document structures, and language-specific evaluation and optimization for regional-language responses. These extensions can improve the retrieval reliability, multilingual robustness, and computational efficiency of DesiQuery.

REFERENCES

- [1] T. F. Khan, M. Arslan, F. Alturise, A. H. Almutairi, and M. T. Raza, "Uqa-rag: Enhancing urdu question-answer retrieval with semantic similarity and llms," *IEEE Access*, vol. 14, pp. 2868–2882, 2026.
- [2] S. Amal, A. R. Niranjana, and M. G. Thushara, "Edurag: A multimodal explainable rag system with deterministic token grounding," *IEEE Access*, vol. 14, pp. 90 849–90 859, 2026.
- [3] M. Hindi, L. Mohammed, O. Maaz, and A. Alwarafy, "Enhancing the precision and interpretability of retrieval-augmented generation (rag) in legal technology: A survey," *IEEE Access*, vol. 13, pp. 46 171–46 189, 2025.
- [4] J. Xu, H. Zhang, H. Zhang, J. Lu, and G. Xiao, "Chattf: A knowledge graph-enhanced intelligent q&a system for mitigating factuality hallucinations in traditional folklore," *IEEE Access*, vol. 12, pp. 162 638–162 650, 2024.
- [5] B. Saha, U. Saha, and M. Zubair Malik, "Quim-rag: Advancing retrieval-augmented generation with inverted question matching for enhanced qa performance," *IEEE Access*, vol. 12, pp. 185 401–185 410, 2024.
- [6] A. Onan, A. H. Nasution, T. Celikten, and P. Cetin, "Comparing chatgpt, gemini, and emerging llms in low-resource educational settings: Reasoning quality, consistency, and explainability," *IEEE Access*, vol. 14, pp. 32 807–32 838, 2026.
- [7] A. Perevalov, A. Gashkov, M. Eltsova, and A. Both, "Sparql query candidate filtering for improving the quality of multilingual question answering over knowledge graphs using language models," *Journal of Web Engineering*, vol. 24, no. 4, pp. 563–592, 2025.
- [8] A. Vijeevaraj Ann Sinthusha, E. Y. A. Charles, and R. Weerasinghe, "Machine reading comprehension for the tamil language with translated squad," *IEEE Access*, vol. 13, pp. 13 312–13 328, 2025.
- [9] M. Ali, R. Speck, H. M. Zahera, M. Saleem, D. Moussallem, and A.-C. Ngonga Ngomo, "Multilingual relation extraction: A survey," *IEEE Access*, vol. 13, pp. 151 907–151 933, 2025.
- [10] K. A. Hambarde and H. Proença, "Information retrieval: Recent advances and beyond," *IEEE Access*, vol. 11, pp. 76 581–76 604, 2023.
- [11] Z. Hu, P. Yang, F. Liu, Y. Meng, and X. Liu, "Prompting large language models with knowledge-injection for knowledge-based visual question answering," *Big Data Mining and Analytics*, vol. 7, no. 3, pp. 843–857, 2024.
- [12] L. Kong, X. Zhong, J. Chen, H. Fu, and Y. Wang, "Multi-perspective consistency checking for large language model hallucination detection: A black-box zero-resource approach," *Frontiers of Information Technology & Electronic Engineering*, vol. 26, no. 11, pp. 2298–2309, 2025.
- [13] S. Behmanesh, A. Talebpour, M. Shamsfard, and M. Mahdi Jafari, "A novel open-domain question answering system on curated and extracted knowledge bases with consideration of confidence scores in existing triples," *IEEE Access*, vol. 12, pp. 160 741–160 760, 2024.
- [14] M. L. Lima de Oliveira and J. de Jesus Fiais Cerqueira, "Resource-constrained evaluation of quantized local llms for retrieval-augmented generation on a reduced-corpus setting," *IEEE Access*, vol. 14, pp. 70 402–70 422, 2026.
- [15] J. Choi, S. Hong, S. Hong, J. Park, and E.-S. Jung, "Toward generating quality test questions and answers using quantized low-rank adapters in llms," *IEEE Access*, vol. 13, pp. 87 793–87 809, 2025.
- [16] P.-J. Liao, H.-Y. Lee, and H.-M. Wang, "Cross-attention reprogramming for asr: Bridging discrete speech units and pretrained language models," *IEEE Access*, vol. 14, pp. 662–678, 2026.



Manishaa Gurunathan <manishaa2408@gmail.com>

Acceptance Letter of NMIC262227_CMT2227

NMIC 2026 <nmic.2026.02@gmail.com>

Sun, Sep 27, 2026 at 10:15 AM

To: Manishaa Gurunathan <manishaa2408@gmail.com>

Acceptance Letter of NMIC262227_CMT2227

Dear Authors,

Congratulations!

We are pleased to share that your article is accepted for presentation and publication in Scopus IEEE Xplore NMIC-2026 with **minor revision**.

Download the registration form from the given link and submit the filled form: https://isve.in/wp-content/uploads/2025/11/Registration-Form_NMIC-2025.pdf within 3-5 days of this mail. Article must be prepared in IEEE double column format: <https://isve.in/wp-content/uploads/2025/11/IEEE-conference-template-a4.docx>. After incorporation of the reviewers comments, submit your final article DOCX, PDF and Plagiarism report in the name of your paper ID (e.g. NMIC26001_CMT1) to reply to this mail nmic.2026.02@gmail.com where the PDF file must be passed through IEEE PDF Express available at CMT Portal and also upload on CMT portal against your submission ID: <https://cmt3.research.microsoft.com/NMIC2026>. Registration has been started whose article reviews are completed and communicated to authors. If facing any problem to convert PDF from IEEE PDF express submit docx file.

Documents required for Registration: Registration Form, Proof of Payments, Article DOCX, PDF, Plagiarism Report etc. Copyright Form will be submitted later. It will be informed by the committee in the due course of time.

Registration information details: <https://isve.in/ieee-nmic-2026-registration-details/>

Conference information details: <https://isve.in/nmic-2026/>

Benefits of IEEE/ ISVE Membership: Authors will get 5% discount in their article fees after the Membership of IEEE/ ISVE. ISVE Membership details are available: <https://isve.in/isve-membership/> interested can fill and submit the form as per their eligibility.

Reviewer comments are given below:-

1. The idea of the article DesiQuery: An Evidence-Grounded Multilingual Framework for Regional Language Question Answering is fine.
2. Author affiliations are given.
3. Number of references is fine.
4. Comparative studies with standard publications like IEEE/ Springer etc. are not shown. Highlight your findings.
5. Formatting of the article is fine.
6. The Figures & Tables numbering and captions are given.

7. Simulation results are shown and explained.
8. References are properly formatted and aligned as per IEEE.
9. Please note that plagiarism should not be more than 10%. Please submit the final paper with a plagiarism report.
10. Send the updated docx and PDF revised file as camera ready paper in the given format <https://isve.in/wp-content/uploads/2025/11/IEEE-conference-template-a4.docx>
11. Articles will be accepted with the above changes.

Thanks & regards